

Integración y calidad de datos para el análisis comercial: prueba de concepto con ETL, data warehouse y dashboard en Apache Superset

Autor: Malquin Pozo, David Sebastián

Tutor: Atudillo Quimbiulco, Kevin Segundo

Facultad de Ingeniería y Ciencias Aplicadas, Universidad Central del Ecuador

Carrera de Sistemas de Información

Trabajo de Titulación modalidad Examen Complexivo teórico-práctico previo a la obtención del título de Ingeniero(a) en Sistemas de Información

Quito, 2026

Datos del componente

Componente: Inteligencia de negocios

Subcomponente: Integración y preparación de datos

Tema específico: Procesos ETL, calidad e integración de fuentes de datos

Propuesta del tema: Atudillo Quimbiulco, Kevin Segundo (C.C. 17230335711)

Autor del informe: Malquin Pozo, David Sebastián (C.C. 1750115089)

Correo institucional: dsmalquin@uce.edu.ec

Tutor: Atudillo Quimbiulco, Kevin Segundo (C.C. 17230335711)

Carrera: Carrera de Sistemas de Información

Repositorio: https://github.com/DavCoder22/Proyecto34

Derechos de autor

Yo Malquin Pozo, David Sebastián, en calidad de autor y titular de los derechos morales y patrimoniales del trabajo de titulación Integración y calidad de datos para el análisis comercial: prueba de concepto con ETL, data warehouse y dashboard en Apache Superset, modalidad Examen Complexivo teórico-práctico, de conformidad con el Art. 114 del CÓDIGO ORGÁNICO DE LA ECONOMÍA SOCIAL DE LOS CONOCIMIENTOS, CREATIVIDAD E INNOVACIÓN, concedo a favor de la Universidad Central del Ecuador una licencia gratuita, intransferible y no exclusiva para el uso no comercial de la obra, con fines estrictamente académicos. Conservo a mi favor todos los derechos de autor sobre la obra, establecidos en la normativa citada.

Asimismo, autorizo a la Universidad Central del Ecuador para que realice la digitalización y publicación de este trabajo de titulación en el repositorio virtual, de conformidad con lo dispuesto en el Art. 144 de la LEY ORGÁNICA DE EDUCACIÓN SUPERIOR.

El autor declara que la obra objeto de la presente autorización es original en su forma de expresión y no infringe el derecho de autor de terceros, asumiendo la responsabilidad por cualquier reclamación que pudiera presentarse por esta causa y liberando a la Universidad de toda responsabilidad.



Firma: _____

Malquin Pozo, David Sebastián

C.C. 1750115089

Dirección electrónica: dsmalquin@uce.edu.ec

Aprobación del tutor

En mi calidad de Tutor del Trabajo de Titulación, presentado por Malquin Pozo, David Sebastián, para optar por el título de Ingeniero(a) en Sistemas de Información; cuyo título es: Integración y calidad de datos para el análisis comercial: prueba de concepto con ETL, data warehouse y dashboard en Apache Superset, considero que dicho trabajo reúne los requisitos y méritos suficientes para ser sometido a la presentación pública y evaluación por parte del tribunal examinador que se designe.

En la ciudad de Quito, 6 de noviembre de 2026.

_____

Atudillo Quimbiulco, Kevin Segundo

DOCENTE-TUTOR

C.C. 17230335711

Tabla de contenidos

Datos del componente....ii

Derechos de autor....iii

Aprobación del tutor....iv

Tabla de contenidos....v

Lista de tablas....viii

Lista de figuras....ix

Lista de anexos....x

Resumen....xi

Abstract....xii

Introducción....1

- Planteamiento del problema....1
- Objetivos....2
- Justificación....3
- Alcance y estructura del documento....3

Teoría....4

- Inteligencia de negocios y analítica....4
- ETL y data warehouse....4
- Calidad de datos....5
- Modelo dimensional....5
- KPIs y dashboards....6
- Apache Superset y tecnologías del stack....6
- Marco metodológico....7

Metodología experimental....7
Enfoque y diseño....7
Fuentes de datos....8
Fases del pipeline ETL....9
Reglas de calidad....12
Modelo dimensional del data warehouse....12
Definición de KPIs....13
Construcción del dashboard....14
Despliegue y reproducibilidad....15
Consideraciones éticas....15
Análisis y resultados....16
Perfilado inicial....16
Comparación de calidad antes y después....16
Registros rechazados....17
Modelo dimensional resultante....18
Indicadores consolidados....19
Evolución mensual de ventas....20
Ventas por categoría....20
Desempeño por sucursal....21
Ventas por segmento....23
Canales y métodos de pago....24
Evidencia del dashboard....25
Cumplimiento de objetivos....28

Auditoría de consistencia de cifras....28
Discusión de resultados....29
Interpretación comercial....29
Contraste con la teoría....30
Limitaciones....31
Validez y amenazas....32
Conclusiones....32
Recomendaciones....33
Referencias....34
Anexos....36
Anexo A. Esquema del modelo estrella....36
Anexo B. Vistas KPI....38
Anexo C. Evidencia de rechazos....47
Anexo D. Reproducibilidad....48
Anexo E. Evidencias del proceso ETL y del dashboard en la nube....49

Lista de tablas

Tabla 1. Fuentes de datos del caso de estudio....8

Tabla 2. Problemas observados y reglas de transformación....12

Tabla 3. Estructura del modelo estrella....13

Tabla 4. Definición de los indicadores del dashboard....13

Tabla 5. Tecnologías utilizadas y función en la solución....15

Tabla 6. Comparación de calidad antes y después del ETL....17

Tabla 7. Registros rechazados por motivo....18

Tabla 8. Indicadores consolidados del universo válido....19

Tabla 9. Ventas por categoría y margen bruto....21

Tabla 10. Desempeño por sucursal....22

Tabla 11. Ventas por segmento de cliente....23

Tabla 12. Canales de venta y métodos de pago....24

Tabla 13. Cumplimiento de los objetivos específicos....28

Lista de figuras

Figura 1. Arquitectura de la solución.....9

Figura 2. Flujo del pipeline ETL.....10

Figura 3. Efecto del proceso de calidad sobre el universo de datos.....17

Figura 4. Evolución mensual de las ventas netas.....20

Figura 5. Participación de las categorías en las ventas netas.....21

Figura 6. Desempeño de las sucursales por región.....22

Figura 7. Participación de los segmentos de cliente.....23

Figura 8. Dashboard Sales Analytics generado desde el data warehouse.....26

Figura 9. Marcas líderes por venta neta.....27

Figura 10. Descuento otorgado por categoría.....27

Figura 11. Pérdida estimada por rechazos según sucursal.....28

Figura 12. Notebook de Jupyter con la ejecución del pipeline ETL.....49

Figura 13. Datos procesados en la etapa de staging.....50

Figura 14. Tablas originales y transformadas cargadas en Neon.....51

Figura 15. Comparación de calidad antes y después del ETL.....51

Figura 16. Reglas de transformación y justificación.....52

Figura 17. Dashboard Sales Analytics en Apache Superset.....53

Figura 18. Modelo analítico actualizado con FactRechazos y vistas nuevas.....54

Figura 19. Dashboard ejecutivo actualizado: KPIs, mapa-filtro y leyenda.....55

Figura 20. Pérdidas por rechazos y descuentos por categoría.....55

Lista de anexos

Anexo A. Esquema del modelo estrella.....36

Anexo B. Vistas KPI.....38

Anexo C. Evidencia de rechazos.....47

Anexo D. Reproducibilidad.....48

Anexo E. Evidencias del proceso ETL y del dashboard en la nube.....49

Integración y calidad de datos para el análisis comercial: prueba de concepto con ETL, data warehouse y dashboard en Apache Superset

Autores: Malquin Pozo, David Sebastián

Tutor: Atudillo Quimbiulco, Kevin Segundo

Resumen

El presente trabajo desarrolla una prueba de concepto para integrar, limpiar, validar y visualizar datos comerciales provenientes de cuatro fuentes heterogéneas: clientes, sucursales, productos y ventas. El problema identificado es la dispersión de la información y la presencia de duplicados, valores nulos, fechas inválidas, categorías con distinta escritura y claves foráneas inexistentes, lo que afecta la confiabilidad de los indicadores. La metodología combinó un pipeline ETL con Python y pandas, un modelo dimensional en esquema estrella y un dashboard en Apache Superset. Se aplicaron reglas de limpieza, homologación, validación referencial y cálculo de ingresos, margen y ticket promedio. Los resultados muestran que el universo inicial de 6.870 líneas de venta se redujo a 6.800 registros válidos, con una cobertura del 98,98 %. El dashboard consolida 7.452.222,71 USD de ingresos netos, 13.817 unidades, 1.966.780,12 USD de margen bruto y 3.587 transacciones. La categoría Accesorios concentra el 20,80 % de las ventas y el segmento Masivo el 50,85 %. Se concluye que la calidad de datos es una condición necesaria para que los indicadores comerciales sean auditables y útiles en la toma de decisiones.

Palabras clave: Inteligencia de negocios, Calidad de datos, ETL, Modelo dimensional

TITLE: Integration and data quality for commercial analysis: a proof of concept with ETL, data warehouse and an Apache Superset dashboard

Authors: Malquin Pozo, David Sebastián

Tutor: Atudillo Quimbiulco, Kevin Segundo

Abstract

This work develops a proof of concept to integrate, clean, validate and visualize commercial data from four heterogeneous sources: customers, branches, products and sales. The identified problem is the dispersion of information and the presence of duplicates, null values, invalid dates, inconsistent category spellings and non-existent foreign keys, which affect the reliability of the indicators. The methodology combined an ETL pipeline with Python and pandas, a star-schema dimensional model and an Apache Superset dashboard. Cleaning, homologation, referential validation and revenue, margin and average ticket calculations were applied. The results show that the initial universe of 6,870 sales lines was reduced to 6,800 valid records, with 98.98 % coverage. The dashboard consolidates USD 7,452,222.71 in net revenue, 13,817 units, USD 1,966,780.12 in gross margin and 3,587 transactions. The Accessories category accounts for 20.80 % of sales and the Masivo segment for 50.85 %. It is concluded that data quality is a necessary condition for commercial indicators to be auditable and useful for decision-making.

Keywords: Business intelligence, Data quality, ETL, Dimensional modeling

Introducción

Las organizaciones comerciales generan datos en múltiples sistemas: registros de clientes, catálogos de productos, información de sucursales y transacciones de venta. Cuando esas fuentes no comparten claves, formatos ni reglas de negocio, los reportes pueden mostrar cifras inconsistentes y las decisiones se apoyan en información parcial. La inteligencia de negocios surge precisamente para convertir esos datos dispersos en información útil para el análisis y la gestión (Sharda et al., 2020). Sin embargo, la visualización por sí sola no corrige los problemas de origen; antes es necesario integrar, limpiar y validar los datos (Kimball & Ross, 2013).

La presente prueba de concepto aborda ese problema en un escenario académico con cuatro fuentes heterogéneas. El proceso construye un pipeline ETL, genera un data warehouse en esquema estrella, calcula ocho indicadores comerciales y de calidad y publica un dashboard interactivo. La evidencia se documenta con reportes de perfilado, comparación antes/después, reglas de transformación y un log de rechazos, de modo que cada cifra publicada pueda rastrearse hasta su origen.

Planteamiento del problema

Las fuentes del caso presentan problemas típicos de integración: 6.870 líneas de venta con 20 duplicados exactos, 30 registros con claves foráneas inexistentes y 20 filas con fechas o valores numéricos inválidos. Además, los catálogos contienen duplicados, valores nulos y categorías escritas de formas distintas. Si esos registros se cargan sin control, el dashboard puede sumar ventas sin cliente, producto o sucursal, o contar dos veces una misma operación. El problema, por tanto, no es solo técnico: afecta la confianza en los indicadores y la calidad de las decisiones comerciales.

La pregunta que orienta el trabajo es la siguiente: ¿cómo integrar y depurar cuatro fuentes comerciales heterogéneas para producir un conjunto de indicadores trazables, reproducibles y útiles para la toma de decisiones? Para responderla se diseña una PoC que prioriza la evidencia cuantificable de calidad y la trazabilidad de cada KPI.

Objetivos

Objetivo general

Diseñar e implementar una prueba de concepto de integración, calidad de datos y visualización comercial que permita obtener indicadores confiables a partir de cuatro fuentes heterogéneas, con trazabilidad completa desde el dato de origen hasta el dashboard.

Objetivos específicos

1. Integrar las fuentes de clientes, sucursales, productos y ventas mediante un pipeline ETL reproducible.
2. Aplicar reglas de limpieza, homologación y validación referencial, documentando los registros rechazados y su motivo.
3. Construir un data warehouse en esquema estrella con dimensiones de tiempo, cliente, producto y sucursal, y una tabla de hechos de ventas.
4. Calcular los indicadores de ingresos netos, unidades vendidas, margen bruto y número de transacciones.
5. Publicar un dashboard interactivo en Apache Superset y una evidencia HTML que permita contrastar los resultados.
6. Verificar la consistencia entre la fuente, el data warehouse y el dashboard, y documentar las limitaciones de la PoC.

Justificación

La PoC se justifica porque demuestra, con datos verificables, que la calidad de datos cambia el universo analítico y la interpretación de los resultados. No se limita a mover archivos: documenta cuántos registros se eliminan, por qué se eliminan y cómo se evita el doble conteo. Esa evidencia es relevante para un tribunal académico y para una mesa de decisiones, porque permite distinguir entre un dato faltante y un dato inválido, y entre una venta real y una venta huérfana.

Adicionalmente, la solución utiliza herramientas abiertas y contenedores, lo que reduce el costo de experimentación y facilita la reproducción en distintos entornos. El diseño puede escalar a producción con un orquestador, una base PostgreSQL gestionada y un gobierno de datos formal, sin reescribir las reglas de negocio principales.

Alcance y estructura del documento

El alcance cubre la integración batch de cuatro fuentes, la construcción del modelo estrella, el cálculo de indicadores comerciales y de calidad y la publicación de un dashboard. No incluye carga incremental, streaming, modelos predictivos, presupuestos por meta ni gobierno de datos corporativo. Los datos son de carácter académico y no representan personas reales; aun así, el diseño prevé la anonimización como requisito para un entorno productivo.

El documento se organiza en ocho secciones. La introducción presenta el problema y los objetivos; la teoría define los conceptos de BI, ETL, calidad, modelo dimensional y dashboards; la metodología describe las fuentes, las fases y las reglas aplicadas; el análisis presenta los resultados; la discusión interpreta los hallazgos y sus limitaciones; las conclusiones y recomendaciones cierran el estudio; finalmente se incluyen las referencias y los anexos técnicos. El formato del documento se apega a los lineamientos de publicación de la Universidad Central

del Ecuador (2022) y a la estructura de la guía metodológica del examen complexivo (Universidad Central del Ecuador, Facultad de Ingeniería y Ciencias Aplicadas, 2026).

Teoría

Inteligencia de negocios y analítica

La inteligencia de negocios (BI) comprende las prácticas, tecnologías y aplicaciones que permiten recopilar, integrar, analizar y presentar información para apoyar la toma de decisiones (Sharda et al., 2020). Su valor no reside únicamente en el reporte final, sino en la capacidad de convertir datos operativos en una representación coherente del negocio. La analítica, por su parte, añade métodos para explorar patrones y generar conocimiento a partir de los datos (Provost & Fawcett, 2013).

En una organización comercial, la BI permite responder preguntas como qué categorías generan más ingresos, qué sucursales tienen mejor desempeño o qué segmentos concentran el volumen. Para que esas respuestas sean válidas, el dato debe ser consistente, completo y trazable. De ahí la importancia de un proceso ETL y de un modelo de datos bien definido.

ETL y data warehouse

El ETL (extract, transform, load) es el proceso que extrae datos de las fuentes, los transforma según reglas de negocio y los carga en un repositorio analítico. Kimball y Ross (2013) plantean que el data warehouse debe organizarse alrededor de procesos de negocio y de un modelo dimensional que facilite la consulta. En ese enfoque, la tabla de hechos almacena las métricas y las dimensiones describen el contexto: quién, qué, cuándo y dónde.

El pipeline de la PoC sigue esa lógica. Primero extrae los archivos originales a una zona de staging; luego perfila, limpia, homologa y valida; después transforma las reglas de negocio y

finalmente carga el modelo estrella en SQLite y, si está configurado, en PostgreSQL. La separación por etapas permite auditar cada paso y repetir el proceso con los mismos insumos.

Calidad de datos

La calidad de datos se evalúa a partir de dimensiones como exactitud, completitud, consistencia, oportunidad y unicidad (Batini et al., 2009; Pipino et al., 2002). La norma ISO 8000-61 propone un modelo de referencia para gestionar la calidad como un proceso, no como una revisión aislada (ISO, 2016). En la práctica, esto implica definir reglas, medir el efecto de cada regla y conservar evidencia de los registros que no cumplen.

La PoC operacionaliza esas dimensiones con controles concretos: eliminación de duplicados exactos, tratamiento de nulos, normalización de fechas y descuentos, homologación de categorías y validación de claves foráneas. Los registros que no superan los controles se excluyen del hecho y se registran en log_rechazos.csv con su motivo. Así, la calidad no se declara: se mide y se documenta.

Modelo dimensional

El modelo estrella es una estructura compuesta por una tabla de hechos central y dimensiones que la rodean. La tabla de hechos contiene las claves foráneas y las métricas del proceso; las dimensiones contienen atributos descriptivos y jerarquías. Este diseño simplifica las consultas analíticas y mejora el rendimiento frente a un modelo normalizado (Kimball & Ross, 2013).

En el caso de estudio, la tabla FactVentas tiene grano de línea de venta, es decir, una fila por producto vendido dentro de una transacción. Las dimensiones DimTiempo, DimCliente, DimProducto y DimSucursal permiten filtrar por fecha, segmento, categoría y sucursal. Las

métricas venta_neta, costo_total y margen se calculan con reglas explícitas para que el dashboard no replique la lógica en cada consulta.

KPIs y dashboards

Un KPI es un indicador cuantificable que refleja el desempeño de un proceso o de un objetivo de negocio. Para que sea útil, debe tener una definición estable, una fuente identificable y una fórmula reproducible. Few (2006) señala que un dashboard efectivo prioriza la información necesaria para decidir y evita el exceso de gráficos sin contexto.

La PoC define cuatro indicadores principales: ingresos netos, unidades vendidas, margen bruto y número de transacciones. A ellos se suman el ticket promedio, el margen porcentual, el descuento otorgado y la pérdida estimada por rechazos. Cada indicador se calcula sobre el mismo universo validado y se complementa con cortes por tiempo, categoría, sucursal, segmento, marca, región y canal. El dashboard incluye filtros y una nota metodológica para recordar que las cifras representan el universo válido, no una estimación de ventas faltantes.

Apache Superset y tecnologías del stack

Apache Superset es una plataforma de BI de código abierto que permite conectar bases de datos, crear datasets, gráficos y dashboards interactivos (Apache Software Foundation, 2026). Su integración con PostgreSQL y su despliegue mediante contenedores la hacen adecuada para una PoC reproducible. El dashboard se complementa con una versión HTML generada desde SQLite, útil como evidencia cuando no se dispone de un servidor de BI.

El procesamiento se realiza con Python y pandas, que ofrecen operaciones vectorizadas para limpieza y transformación (pandas Development Team, 2026). PostgreSQL actúa como motor del data warehouse para Superset (PostgreSQL Global Development Group, 2026),

mientras que SQLite sirve como respaldo local. Docker facilita la ejecución portable (Docker Inc., 2026) y Northflank permite publicar los servicios en la nube (Northflank, 2026).

Marco metodológico

El trabajo se apoya en un enfoque aplicado, cuantitativo y descriptivo. La metodología de investigación guía la formulación del problema, la recolección de evidencia y la interpretación de resultados (Hernández-Sampieri & Mendoza, 2018). Para el ciclo de vida de los datos se toma como referencia CRISP-DM, que organiza el trabajo en comprensión del negocio, comprensión de los datos, preparación, modelado, evaluación y despliegue (Wirth & Hipp, 2000).

La adaptación de CRISP-DM a esta PoC es directa: el negocio se traduce en indicadores comerciales y de calidad; los datos se perfilan; la preparación corresponde al ETL; el modelado es el esquema estrella; la evaluación se realiza con comparaciones antes/después y controles de totales; y el despliegue se concreta en Docker, Northflank y Superset. Esta correspondencia permite justificar cada etapa con evidencia verificable.

Metodología experimental

Enfoque y diseño

La investigación es aplicada, cuantitativa y descriptiva. Es aplicada porque resuelve un problema concreto de integración y análisis comercial; es cuantitativa porque mide registros, proporciones e indicadores; y es descriptiva porque caracteriza el estado de las fuentes y el efecto de las reglas de calidad. El diseño es no experimental y transversal: no se manipulan variables del negocio, sino que se observa el resultado del pipeline sobre un corte de datos de enero de 2024 a agosto de 2026.

La unidad de análisis es la línea de venta. Cada fila de FactVentas representa un producto dentro de una transacción; por eso una misma venta puede tener varias líneas. Esta decisión es

importante: si se contara el número de filas como número de ventas, se sobreestimaría el indicador de transacciones. El conteo de transacciones se realiza con COUNT(DISTINCT id_venta), mientras que los ingresos, las unidades y el margen se suman por línea.

El procedimiento se ejecutó en seis momentos: caracterización de fuentes, definición de reglas, implementación del ETL, construcción del modelo estrella, cálculo y publicación de KPIs, y verificación de consistencia. Todos los scripts y reportes quedaron versionados en el repositorio, de modo que el resultado pueda reproducirse con los mismos archivos de entrada.

Fuentes de datos

Se utilizaron cuatro fuentes: clientes y sucursales en CSV, productos en Excel y ventas en CSV. La Tabla 1 resume el volumen y la función de cada una. Las ventas son la fuente principal del hecho; las demás alimentan las dimensiones y permiten segmentar los resultados.

Tabla 1

Fuentes de datos del caso de estudio

Fuente	Archivo	Tipo	Filas	Clave	Uso
Clientes	clientes.csv	CSV	1.006	cod_cliente	Dimensión cliente y segmentación
Sucursales	sucursales.csv	CSV	12	cod_sucursal	Dimensión sucursal y región
Productos	productos.xlsx	Excel	123	cod_producto	Dimensión producto, categoría y costo
Ventas	ventas_sucursales.csv	CSV	6.870	id_venta	Tabla de hechos a nivel de línea de venta

Nota. Elaboración propia a partir de los archivos de 01_datos/raw. Las filas corresponden a los registros originales, antes de la limpieza.

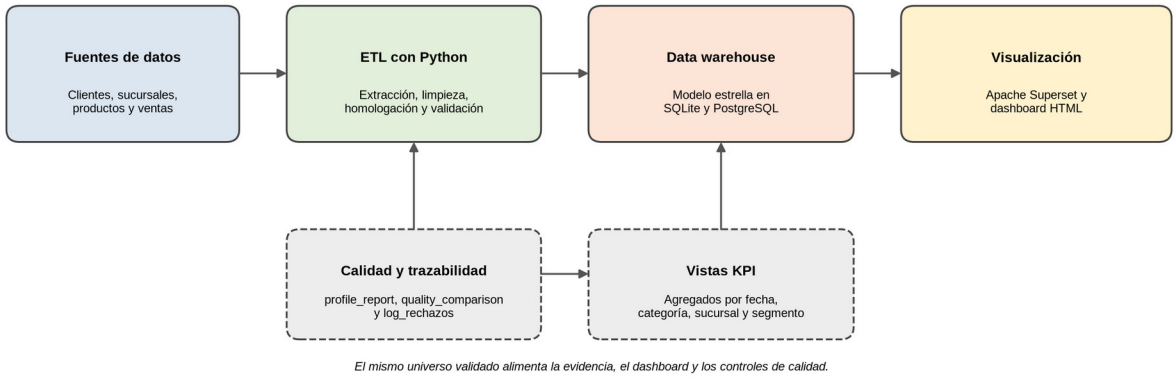
La fuente de ventas contiene 6.870 líneas y 3.600 identificadores únicos de venta antes de la limpieza, lo que confirma que el grano es la línea y no la transacción. El catálogo de productos tiene 123 filas y 120 claves únicas; el de clientes, 1.006 filas y 1.000 claves; y el de sucursales, 12 filas sin duplicados. Esas diferencias son la primera señal de que los catálogos requieren depuración antes de relacionarse con el hecho.

La Figura 1 resume la arquitectura de la solución. Las fuentes alimentan un ETL que deja evidencia de calidad, carga un data warehouse en esquema estrella y publica los resultados en

Superset y en un dashboard HTML. El mismo universo validado se utiliza para el análisis, los controles y la visualización.

Figura 1

Arquitectura de la solución



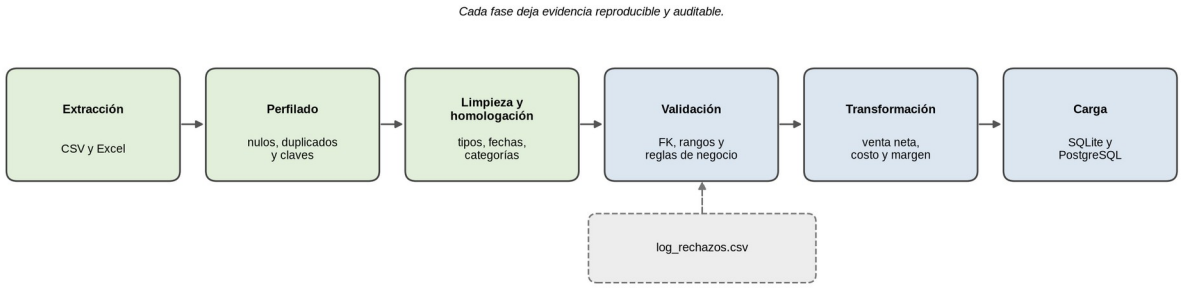
Nota. Elaboración propia a partir de 02_etl/, 03_dw/, 04_dashboard/ y docker/.

Fases del pipeline ETL

El pipeline se implementó en Python con módulos separados por responsabilidad: extract.py, profile.py, clean.py, homologate.py, validate.py, transform.py, load.py y main.py. La separación permite probar cada fase y reemplazarla sin reescribir todo el proceso. La Figura 2 muestra la secuencia general.

Figura 2

Flujo del pipeline ETL



Nota. Elaboración propia a partir de 02_etl/main.py. La validación alimenta el log de rechazos y separa los registros que no pueden formar parte del hecho.

Extracción

La extracción copia los archivos originales a la zona de staging y los lee con pandas. Para CSV se usa codificación UTF-8 con BOM y para Excel se lee la primera hoja. En esta fase no se modifica el contenido: se conserva una copia del dato original para poder comparar antes y después. La evidencia de perfilado se genera antes de cualquier transformación.

Perfilado

El perfilado describe filas, columnas, claves únicas, valores nulos, duplicados exactos y distribución de valores categóricos. Su objetivo es detectar problemas antes de decidir una regla. Por ejemplo, el perfilado de clientes mostró 1.006 filas con 1.000 claves únicas y 10 valores nulos en segmento; el de productos, 123 filas con 120 claves y 5 marcas nulas; y el de ventas, 6.870 filas con 3.600 claves únicas y 27 nulos repartidos entre canal y método de pago.

Limpieza

La limpieza normaliza cadenas, elimina espacios, convierte valores vacíos en nulos y ajusta tipos de datos. En ventas se eliminan duplicados exactos, se interpretan las fechas con pandas.to_datetime y se convierten cantidad, precio y descuento a numérico. En productos se

convierte el costo unitario. En clientes y sucursales se normalizan fechas. La limpieza no imputa valores de negocio: cuando un dato no existe, se deja como nulo o se reemplaza por una categoría explícita de no especificado, según la regla definida.

Homologación

La homologación unifica variantes de escritura. En categorías, TECNOLOGÍA, tecnologia y tecnología se transforman en Tecnología. En segmento, los valores vacíos se convierten en Sin segmento. En canal y método de pago, los vacíos se convierten en No especificado. Esta fase evita que una misma categoría aparezca como varias series en el dashboard y hace visible la incertidumbre en lugar de ocultarla.

Validación y rechazo

La validación comprueba que las claves foráneas de ventas existan en clientes, productos y sucursales. También verifica que la fecha sea válida, que la cantidad sea mayor que cero, que el precio no sea negativo y que el descuento esté entre 0 y 1. Las filas que no cumplen se excluyen del hecho y se registran en log_rechazos.csv con el motivo. Esta decisión evita ventas huérfanas y permite monitorear la tasa de rechazo como un indicador de calidad del origen.

Transformación

La transformación aplica las reglas de negocio: venta_bruta = cantidad × precio_unitario; venta_neta = venta_bruta × (1 − descuento); costo_total = cantidad × costo_unitario; y margen = venta_neta − costo_total. El descuento se normaliza antes de calcular: si un valor es mayor que 1, se interpreta como porcentaje y se divide para 100. Cuando el costo de un producto es desconocido, no se imputa cero, porque eso sobreestimaría el margen; el margen queda como desconocido y se excluye del agregado correspondiente.

Carga

La carga escribe el modelo estrella en SQLite y, si la variable DATABASE_URL está definida, también en PostgreSQL. Las vistas KPI se recrean en cada ejecución para que Superset y el dashboard HTML consuman el mismo universo validado. La carga es idempotente: si se vuelve a ejecutar el pipeline, las tablas se reemplazan y los resultados son los mismos para los mismos insumos.

Reglas de calidad

La Tabla 2 resume los problemas observados, la fuente afectada, la regla aplicada y el efecto esperado. Las reglas se diseñaron para preservar la mayor cantidad de información posible sin aceptar registros que distorsionen los agregados. Cuando una fila no se puede integrar, se rechaza y se documenta; no se fuerza su inclusión.

Tabla 2

Problemas observados y reglas de transformación

Problema	Fuente	Regla aplicada	Efecto esperado
Duplicados exactos	Las cuatro fuentes	Eliminar filas idénticas; en catálogos conservar una fila por clave natural	Evitar doble conteo y claves repetidas
Fechas heterogéneas o inválidas	Clientes, sucursales y ventas	Parsear con pandas.to_datetime; excluir ventas sin fecha válida	Dimensión tiempo con fechas válidas y agrupación mensual confiable
Descuentos en decimal o porcentaje	Ventas	Valores mayores que 1 se dividen para 100	Descuento entre 0 y 1 para calcular venta neta
Categorías con distinta escritura	Productos	Homologar mayúsculas, minúsculas y acentos a un catálogo canónico	Una sola serie por categoría en el dashboard
Segmento, canal o método vacío	Clientes y ventas	Reemplazar vacío por una categoría explícita de no especificado	No perder filas y hacer visible la incertidumbre
Claves foráneas inexistentes	Ventas	Registrar el rechazo y excluir la línea del hecho integrado	Impedir ventas huérfanas que distorsionen filtros y agregados
Costo de producto desconocido	Productos	No imputar cero; conservar el margen como desconocido	No sobreestimar artificialmente el margen bruto
Tipos numéricos almacenados como texto	Ventas y productos	Convertir cantidad, precio, descuento y costo a numérico	Cálculos reproducibles de ingresos y margen

Nota. Elaboración propia a partir de 06_calidad/reglas_transformacion.md y de la implementación en 02_etl/.

Modelo dimensional del data warehouse

El modelo estrella resultante tiene una tabla de hechos y cuatro dimensiones. La Tabla 3 describe el grano, las claves y las métricas principales. La dimensión tiempo usa una clave

inteligente con formato AAAAMMDD; las dimensiones de cliente, producto y sucursal usan claves sustitutas enteras para desacoplar el modelo de las claves naturales del origen.

Tabla 3

Estructura del modelo estrella

Tabla	Grano	Claves	Atributos o métricas
FactVentas	Línea de venta	id_venta_pk; id_tiempo, id_cliente, id_producto, id_sucursal	cantidad, precio_unitario, descuento_pct, venta_bruta, venta_neta, costo_total, margen
DimTiempo	Día	id_tiempo	fecha, anio, mes, mes_nombre, trimestre, semana, dia_semana
DimCliente	Cliente	id_cliente; cod_cliente	nombre, segmento, ciudad, provincia, fecha_registro, estado
DimProducto	Producto	id_producto; cod_producto	nombre, categoria, subcategoria, marca, costo_unitario, estado
DimSucursal	Sucursal	id_sucursal; cod_sucursal	nombre, ciudad, provincia, region, fecha_apertura, estado

Nota. Elaboración propia a partir de 03_dw/schema.sql y 03_dw/schema_sqlite.sql.

Definición de KPIs

Los indicadores principales se definen sobre FactVentas para garantizar que todos los cortes utilicen la misma base. La Tabla 4 presenta la expresión SQL y la interpretación de cada indicador. El ticket promedio se calcula por transacción, no por línea: primero se suman las líneas de cada id_venta y luego se promedia, para no sesgar el resultado hacia ventas con más productos. El descuento otorgado y la pérdida estimada por rechazos se incorporan como indicadores complementarios de gestión comercial y de calidad.

Tabla 4

Definición de los indicadores del dashboard

Indicador	Expresión	Interpretación
Ingresos netos	SUM(venta_neta)	Dinero efectivamente reconocido después del descuento
Unidades vendidas	SUM(cantidad)	Volumen físico de productos vendidos
Margen bruto	SUM(margen)	Diferencia entre venta neta y costo de los productos
Transacciones	COUNT(DISTINCT id_venta)	Número de operaciones comerciales, no de líneas
Ticket promedio	SUM(venta_neta) / COUNT(DISTINCT id_venta)	Venta neta media por operación
Margen %	SUM(margen) / SUM(venta_neta)	Rentabilidad relativa sobre ingresos netos
Descuento otorgado	SUM(venta_bruta − venta_neta)	Monto cedido por descuentos; se analiza por categoría y provincia
Pérdida por rechazos	SUM(venta_perdida) en fact_rechazos	Venta estimada que no entró al hecho; se monitorea por sucursal y fecha

Nota. Elaboración propia a partir de 04_dashboard/kpi_views_postgres.sql y del notebook 02_etl/01_etl_pipeline.ipynb.

Además de los ocho indicadores, el dashboard presenta cortes por categoría, sucursal, segmento, marca, región, canal y método de pago. Cada corte se calcula sobre el mismo universo y se expresa como participación porcentual sobre los ingresos netos. El mapa provincial funciona como filtro: al seleccionar una provincia o una sucursal, el resto de gráficos se recalcula; al seleccionar una categoría en el pastel, el mapa muestra solo las sucursales donde se vende esa categoría y colorea por intensidad de venta.

Construcción del dashboard

El dashboard Sales Analytics se organiza en capas de decisión. La primera fila contiene ocho tarjetas de KPI: ingresos netos, unidades, margen bruto, descuento otorgado, transacciones, ticket promedio, margen porcentual y pérdida por rechazos. La segunda fila combina el mapa provincial con el pastel de participación por categoría. Debajo se presenta la evolución mensual y, luego, los cortes de categoría, sucursal, segmento, marca, descuento, región y canal. La última sección muestra los líderes (marca, categoría y sucursal) y las pérdidas por sucursal y por fecha.

El mapa es el filtro principal. Los polígonos representan provincias y los puntos rojos, sucursales. Al hacer clic en una provincia o en una sucursal, Superset emite un filtro cruzado y todos los gráficos se recalculan. Al seleccionar una categoría en el pastel, el mapa se filtra y colorea por la intensidad de venta de esa categoría, de modo que se identifican las sucursales donde se comercializa. La leyenda explica la escala de color y el borde regional: turquesa para la Costa, café para la Sierra y verde para la Amazonía.

La construcción sigue tres pasos. Primero se crean las vistas KPI en la base de datos, incluida la vista de detalle que comparte dimensiones para habilitar el filtrado cruzado. Segundo se definen los datasets y gráficos en Superset. Tercero se valida que los totales del dashboard

coincidan con la consulta de control sobre FactVentas. Esa validación es la que convierte el dashboard en evidencia y no solo en una ilustración.

Despliegue y reproducibilidad

La solución puede ejecutarse de forma local o en la nube. El código fuente y los entregables están disponibles en el repositorio https://github.com/DavCoder22/Proyecto34. En local, el comando run_local.bat o docker compose levanta PostgreSQL, Jupyter y Superset. En la nube, el job ETL de Northflank carga los datos en Neon y el servicio Superset los publica en el puerto 8008. El ETL se ejecuta mediante el job dedicado o al arrancar Jupyter; después, Superset consume las tablas y vistas del data warehouse.

La reproducibilidad se apoya en cuatro elementos: archivos de entrada versionados, scripts con rutas relativas, variables de entorno para credenciales y un log de ejecución con los conteos de duplicados, rechazos y filas del hecho. Cualquier persona con el repositorio (https://github.com/DavCoder22/Proyecto34) y las dependencias puede repetir el proceso y obtener las mismas cifras. La Tabla 5 resume las tecnologías y su función.

Tabla 5

Tecnologías utilizadas y función en la solución

Capa	Tecnología	Función
Extracción y transformación	Python + pandas	Lectura de CSV/Excel, limpieza, homologación y cálculo de métricas
Data warehouse local	SQLite	Evidencia reproducible y respaldo del dashboard HTML
Data warehouse en BI	PostgreSQL	Almacenamiento del modelo estrella para Superset
Visualización	Apache Superset	Dashboard interactivo con mapa-filtro, ocho KPIs y filtros cruzados
Evidencia HTML	HTML + JavaScript	Versión portable del dashboard generada desde SQLite
Contenedores	Docker / Docker Compose	Ejecución portable de Jupyter, PostgreSQL y Superset
Nube	Northflank + Neon	Job ETL y servicio Superset en el puerto 8008

Nota. Elaboración propia a partir de requirements.txt, docker/ y deploy/SUPERSET_DEPLOY.md.

Consideraciones éticas

Los datos utilizados son de carácter académico y no corresponden a personas reales. Aun así, el diseño contempla la minimización de datos, la anonimización de nombres y la restricción

de acceso a credenciales. Las variables de entorno se mantienen fuera del repositorio y las contraseñas de ejemplo no deben reutilizarse en producción. La publicación de resultados agregados evita exponer información individual.

Análisis y resultados

Perfilado inicial

El perfilado se realizó antes de cualquier transformación. El periodo analizado comprende del 2024-01-01 al 2026-08-31, con 32 meses. La fuente de ventas tenía 6.870 líneas, 3.600 identificadores únicos y 27 valores nulos; clientes tenía 1.006 filas, 1.000 claves únicas y 10 nulos en segmento; productos tenía 123 filas, 120 claves únicas y 5 marcas nulas; y sucursales tenía 12 filas completas. Los duplicados exactos fueron 20 en ventas, 6 en clientes y 3 en productos.

El perfilado también mostró variantes de escritura en categorías (por ejemplo, TECNOLOGÍA frente a Tecnología) y en ciudades (QUITO, Quito y quito). Esos hallazgos justificaron una fase de homologación antes de la validación, porque de lo contrario una misma categoría se habría dividido en varias series y habría distorsionado la participación porcentual.

Comparación de calidad antes y después

La Tabla 6 compara el estado de las entidades antes y después del ETL. En ventas, la diferencia de 70 filas se explica por 20 duplicados exactos, 30 registros con claves foráneas inválidas y 20 filas con fecha o valores numéricos inválidos. En clientes se eliminaron 6 duplicados y en productos 3. El total de duplicados exactos en las cuatro fuentes fue 29: 20 en ventas y 9 en los catálogos. Las 70 filas excluidas del hecho corresponden solo a ventas: 20 duplicados, 30 claves inválidas y 20 fechas o valores inválidos.

Tabla 6

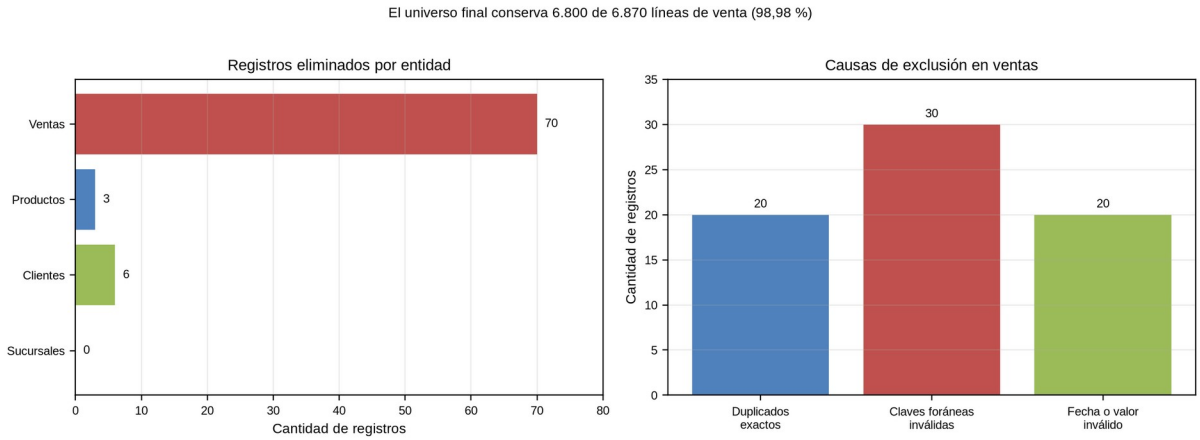
Comparación de calidad antes y después del ETL

Entidad	Filas antes	Filas después	Diferencia	Nulos antes	Nulos después	Duplicados	Claves únicas
Clientes	1.006	1.000	6	10	0	6	1.000
Sucursales	12	12	0	0	0	0	12
Productos	123	120	3	5	5	3	120
Ventas	6.870	6.800	70	27	0	20	3.587

Nota. Elaboración propia a partir de 06_calidad/quality_comparison.csv. Los duplicados corresponden a filas exactas antes de la limpieza.

Figura 3

Efecto del proceso de calidad sobre el universo de datos



Nota. Elaboración propia con los datos de 06_calidad/quality_comparison.csv y 06_calidad/log_rechazos.csv. La cobertura final es 98,98 %.

El resultado más importante es que la reducción no fue arbitraria: cada fila excluida tiene una causa documentada. Las ventas pasaron de 6.870 a 6.800 líneas, con una cobertura de 98,98 %. Los nulos de ventas se eliminaron porque las filas afectadas también carecían de fecha o de valores numéricos válidos, o porque correspondían a duplicados. En clientes quedaron 8 nulos de segmento, que se homologaron como Sin segmento; en productos quedaron 5 marcas nulas, que no impiden el análisis por categoría.

Registros rechazados

El log de rechazos contiene 50 registros: 30 por claves foráneas inexistentes y 20 por fecha o valores numéricos inválidos. Las claves foráneas se reparten en 12 clientes, 10 productos

y 8 sucursales inexistentes. La Tabla 7 resume los motivos. Estos registros no se imputan ni se fuerzan: se excluyen del hecho y se conservan como evidencia para que el área de origen corrija la causa.

Tabla 7

Registros rechazados por motivo

Motivo	Registros	Tratamiento
Regla de validez: fecha o valor numerico invalido	20	Excluido del hecho y registrado en log_rechazos.csv
FK invalida: cod_cliente no existe en cod_cliente	12	Excluido del hecho y registrado en log_rechazos.csv
FK invalida: cod_producto no existe en cod_producto	10	Excluido del hecho y registrado en log_rechazos.csv
FK invalida: cod_sucursal no existe en cod_sucursal	8	Excluido del hecho y registrado en log_rechazos.csv
Total de rechazos	50	Tasa de rechazo: 0,73 % sobre 6.850 filas después de deduplicar

Nota. Elaboración propia a partir de 06_calidad/log_rechazos.csv. La tasa de rechazo se calcula sobre las filas que quedan después de eliminar duplicados exactos.

La tasa de rechazo es un indicador que debe monitorearse. Si aumenta o se concentra en una sucursal o categoría, la tendencia observada puede quedar sesgada. Para no perder esa información, los rechazos se cargan en la tabla fact_rechazos con su motivo y una pérdida estimada de 44.001,54 USD, calculada como cantidad × precio × (1 − descuento). Así, el dashboard puede mostrar las pérdidas por sucursal y por fecha sin confundirlas con ventas válidas.

Modelo dimensional resultante

El data warehouse quedó conformado por 6.800 filas en FactVentas, 1.000 clientes, 120 productos, 12 sucursales y 952 días con ventas en DimTiempo. La diferencia entre 3.600 identificadores únicos de venta en la fuente y 3.587 en el hecho se explica por los rechazos: algunas ventas tenían todas sus líneas inválidas o compartían identificador con registros duplicados. El hecho conserva 6.800 líneas, que equivalen a 1,90 líneas por transacción en promedio.

El modelo permite responder consultas analíticas sin repetir la lógica de negocio. Por ejemplo, la venta neta ya está calculada en el hecho; el margen ya descuenta el costo; y las dimensiones permiten agrupar por mes, categoría, sucursal o segmento. Esa separación es la que hace que Superset pueda crear gráficos con filtros cruzados sin recalcular las reglas en cada visualización.

Indicadores consolidados

La Tabla 8 presenta los indicadores del universo válido. Los ingresos netos ascienden a 7.452.222,71 USD; las unidades vendidas, a 13.817; el margen bruto, a 1.966.780,12 USD; y las transacciones, a 3.587. El margen representa 26,39% de los ingresos y el ticket promedio por transacción es 2.077,56 USD. El descuento otorgado suma 404.749,54 USD y la pérdida estimada por rechazos, 44.001,54 USD.

Tabla 8

Indicadores consolidados del universo válido

Indicador	Expresión	Resultado
Ingresos netos	SUM(venta_neta)	7.452.222,71
Unidades vendidas	SUM(cantidad)	13.817
Margen bruto	SUM(margen)	1.966.780,12
Transacciones	COUNT(DISTINCT id_venta)	3.587
Ticket promedio	ingresos / transacciones	2.077,56
Margen %	margen / ingresos	26,39%
Descuento otorgado	SUM(venta_bruta − venta_neta)	404.749,54
Pérdida por rechazos	SUM(venta_perdida) en fact_rechazos	44.001,54

Nota. Elaboración propia a partir de la consulta de control sobre FactVentas en 03_dw/dw.db y de 06_calidad/log_rechazos.csv. Cifras en USD y unidades.

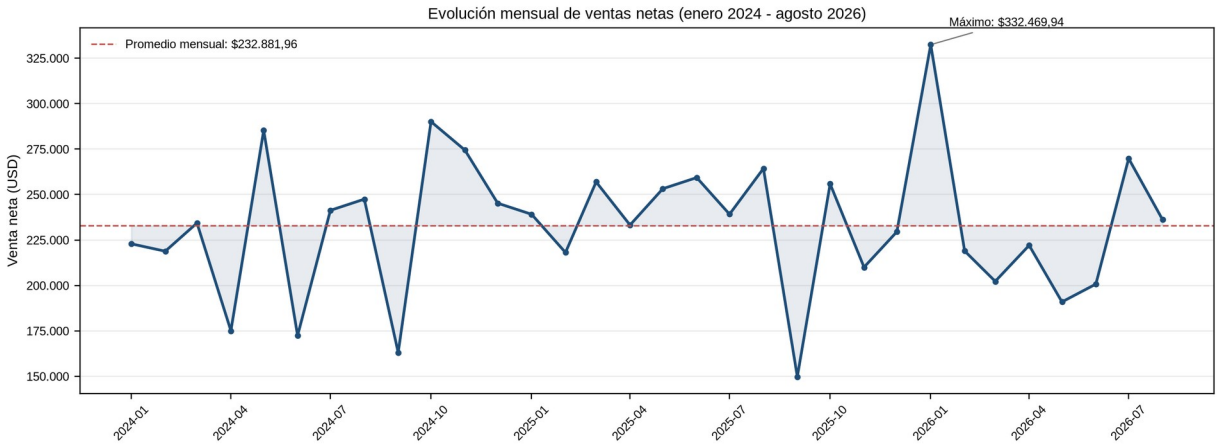
El ticket promedio se calculó como ingresos netos divididos para transacciones, no para líneas. Si se dividiera para las 6.800 líneas, el resultado sería 1.095,92 USD y estaría subestimado, porque una transacción con varios productos aporta varias líneas. Esta distinción es metodológica y debe explicarse en la mesa de decisiones para evitar comparaciones incorrectas.

Evolución mensual de ventas

La Figura 4 muestra la serie mensual de venta neta para los 32 meses analizados. El promedio mensual es 232.881,96 USD y el máximo se registra en enero de 2026, con 332.469,94 USD. Se observan caídas en septiembre de 2024 y septiembre de 2025, seguidas de recuperación en los meses posteriores; ese patrón sugiere estacionalidad y justifica un análisis específico por campaña y por categoría.

Figura 4

Evolución mensual de las ventas netas



Nota. Elaboración propia a partir de la vista v_ventas_mes en 03_dw/dw.db. La línea punteada representa el promedio mensual.

La serie no debe interpretarse como una proyección. Es una descripción del universo válido y su cobertura es alta, pero no incorpora carga incremental ni corrige los rechazos. Para proyectar, sería necesario modelar la estacionalidad, controlar la tasa de rechazo y validar la cobertura de fechas en cada periodo.

Ventas por categoría

La categoría Accesorios lidera con 1.550.022,43 USD, equivalentes al 20,80% de los ingresos. Le siguen Hogar, con 1.459.739,31 USD (19,59%), y Tecnología, con 1.307.815,06

USD (17,55%). Accesorios y Hogar concentran el 40,39 % de las ventas, lo que sugiere priorizar inventario y campañas en esas categorías antes que en las de menor participación.

Tabla 9

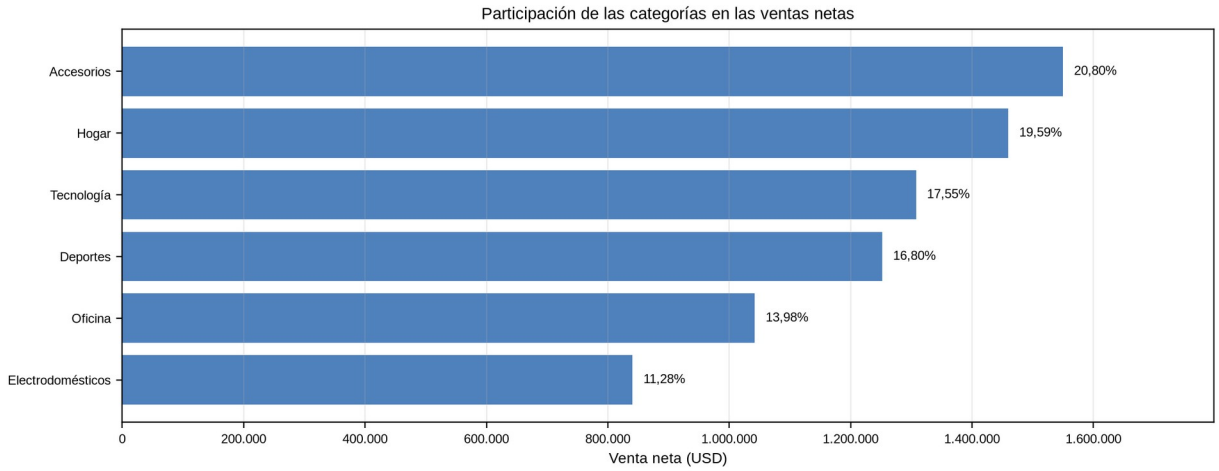
Ventas por categoría y margen bruto

Categoría	Venta neta (USD)	Participación	Margen bruto (USD)
Accesorios	1.550.022,43	20,80%	396.930,25
Hogar	1.459.739,31	19,59%	397.148,96
Tecnología	1.307.815,06	17,55%	343.203,84
Deportes	1.251.944,47	16,80%	325.538,34
Oficina	1.042.055,77	13,98%	274.222,29
Electrodomésticos	840.645,67	11,28%	229.736,44

Nota. Elaboración propia a partir de v_ventas_categoria y del cruce con DimProducto en 03_dw/dw.db.

Figura 5

Participación de las categorías en las ventas netas



Nota. Elaboración propia a partir de v_ventas_categoria. Los porcentajes se calculan sobre los ingresos netos totales.

El margen bruto por categoría no sigue exactamente el orden de las ventas. Hogar aporta 397.148,96 USD de margen, ligeramente por encima de Accesorios (396.930,25 USD), aunque Accesorios vende más. Esa diferencia muestra por qué conviene analizar ingresos y margen en conjunto: una categoría puede liderar ventas y no ser la más rentable.

Desempeño por sucursal

La sucursal con mayor venta neta es Guayaquil Centro, con 686.665,86 USD (9,21%). Le siguen Guayaquil Norte y Loja. La diferencia entre la primera y la última sucursal es de

aproximadamente 132.851 USD, lo que refleja una distribución relativamente homogénea, aunque con oportunidades de mejora en las sucursales de menor volumen. La región Sierra concentra 57,99% de los ingresos y la Costa el 42,01%.

Tabla 10

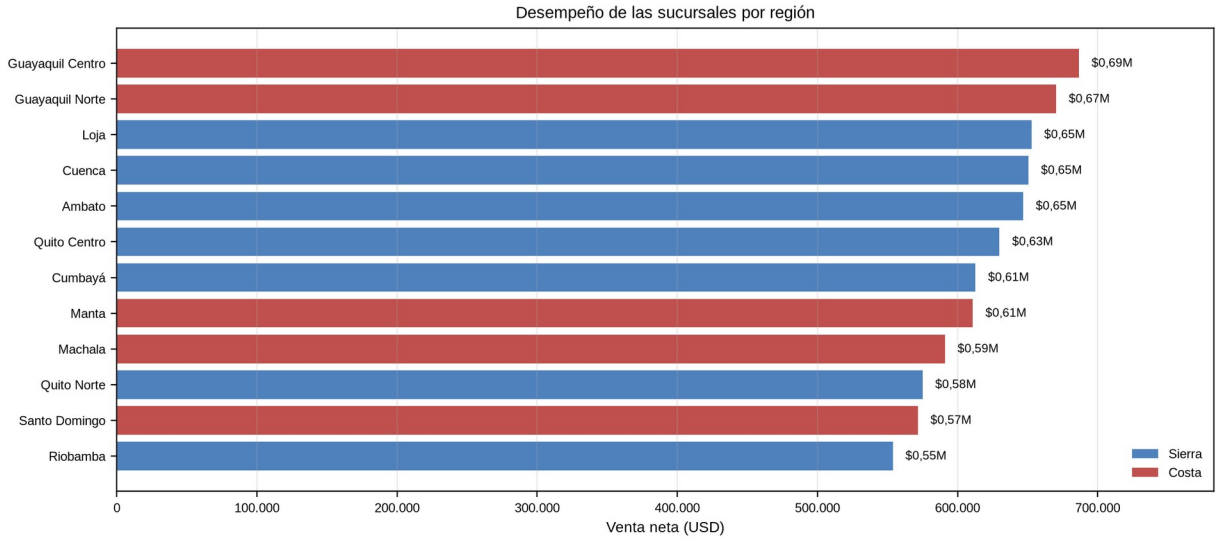
Desempeño por sucursal

Sucursal	Región	Venta neta (USD)	Participación	Transacciones	Ticket promedio (USD)
Guayaquil Centro	Costa	686.665,86	9,21%	334	2.055,89
Guayaquil Norte	Costa	670.333,71	9,00%	287	2.335,66
Loja	Sierra	652.772,23	8,76%	316	2.065,73
Cuenca	Sierra	650.496,32	8,73%	296	2.197,62
Ambato	Sierra	646.825,80	8,68%	274	2.360,68
Quito Centro	Sierra	629.929,48	8,45%	334	1.886,02
Cumbayá	Sierra	612.780,84	8,22%	317	1.933,06
Manta	Costa	610.913,12	8,20%	285	2.143,55
Machala	Costa	591.013,26	7,93%	311	1.900,36
Quito Norte	Sierra	575.037,96	7,72%	295	1.949,28
Santo Domingo	Costa	571.639,53	7,67%	254	2.250,55
Riobamba	Sierra	553.814,61	7,43%	284	1.950,05

Nota. Elaboración propia a partir de v_ventas_sucursal. El ticket promedio se calcula por transacción.

Figura 6

Desempeño de las sucursales por región



Nota. Elaboración propia a partir de v_ventas_sucursal. Azul: Sierra; rojo: Costa.

El ticket promedio más alto corresponde a Ambato (2.360,68 USD), seguido de Guayaquil Norte (2.335,66 USD) y Santo Domingo (2.250,55 USD). Guayaquil Centro, en

cambio, tiene el mayor volumen pero un ticket de 2.055,89 USD. Esa combinación sugiere que el volumen no siempre acompaña al ticket y que las acciones comerciales deben diferenciarse: en las sucursales de mayor ticket conviene proteger la venta cruzada; en las de mayor volumen, aumentar el valor promedio por operación.

Ventas por segmento

El segmento Masivo concentra 3.789.241,73 USD, equivalentes al 50,85% de los ingresos, con 1.846 transacciones. Le sigue Frecuente, con 2.120.524,67 USD (28,45%), y Premium, con 1.088.077,36 USD (14,60%). El segmento Corporativo aporta 4,96 % y el grupo Sin segmento 1,14 %, lo que evidencia el efecto de los nulos homologados.

Tabla 11

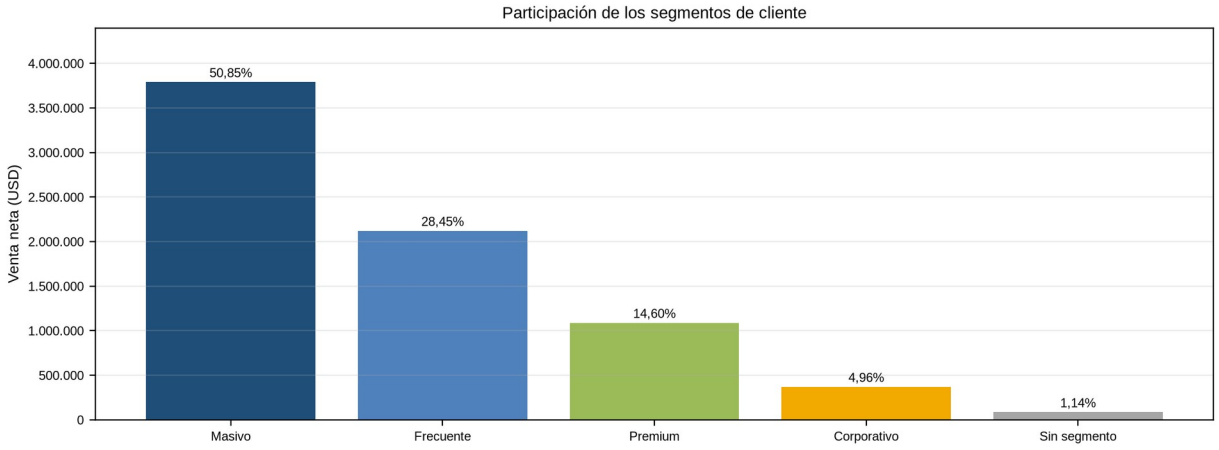
Ventas por segmento de cliente

Segmento	Venta neta (USD)	Participación	Transacciones	Ticket promedio (USD)
Masivo	3.789.241,73	50,85%	1.846	2.052,68
Frecuente	2.120.524,67	28,45%	992	2.137,63
Premium	1.088.077,36	14,60%	532	2.045,26
Corporativo	369.686,60	4,96%	181	2.042,47
Sin segmento	84.692,35	1,14%	36	2.352,57

Nota. Elaboración propia a partir de v_ventas_segmento. El segmento Sin segmento agrupa los valores nulos homologados.

Figura 7

Participación de los segmentos de cliente



Nota. Elaboración propia a partir de v_ventas_segmento. Los porcentajes se calculan sobre los ingresos netos totales.

El ticket promedio más alto se observa en el grupo Sin segmento (2.352,57 USD), seguido de Frecuente (2.137,63 USD). El segmento Masivo, aunque concentra el volumen, tiene un ticket de 2.052,68 USD. Esto sugiere que las campañas de fidelización en Frecuente podrían aumentar la recurrencia sin sacrificar valor por operación, mientras que en Masivo el foco debería estar en el volumen y en la venta cruzada.

Canales y métodos de pago

Los canales de venta están distribuidos de forma equilibrada: Marketplace, Web, Aplicación móvil y Tienda física aportan entre 24,31 % y 25,44 % cada uno. El canal No especificado representa apenas 0,21 %. En métodos de pago, Efectivo lidera con 21,11 %, seguido de Tarjeta de débito (20,38 %), Billetera digital (20,09 %), Tarjeta de crédito (19,31 %) y Transferencia (18,89 %). Esa distribución indica que la operación no depende de un único canal ni de un único medio de pago.

Tabla 12

Canales de venta y métodos de pago

Dimensión	Categoría	Venta neta (USD)	Participación
Canal	Marketplace	1.896.060,65	25,44%
	Web	1.890.436,81	25,37%
	Aplicación móvil	1.838.348,78	24,67%
	Tienda física	1.811.538,54	24,31%
	No especificado	15.837,94	0,21%
Método de pago	Efectivo	1.573.391,40	21,11%
	Tarjeta de débito	1.518.648,59	20,38%
	Billetera digital	1.497.078,91	20,09%
	Tarjeta de crédito	1.438.890,16	19,31%
	Transferencia	1.407.592,20	18,89%
	No especificado	16.621,45	0,22%

Nota. Elaboración propia a partir de las vistas v_ventas_canal y de la consulta agrupada por metodo_pago en 03_dw/dw.db.

El producto con mayor venta neta es Transporte Modelo 055, de la categoría Accesorios, con 148.826,24 USD. Le siguen dos productos de Accesorios, uno de Oficina y uno de Tecnología. La concentración en productos específicos refuerza la necesidad de analizar el surtido y el inventario, no solo la categoría agregada.

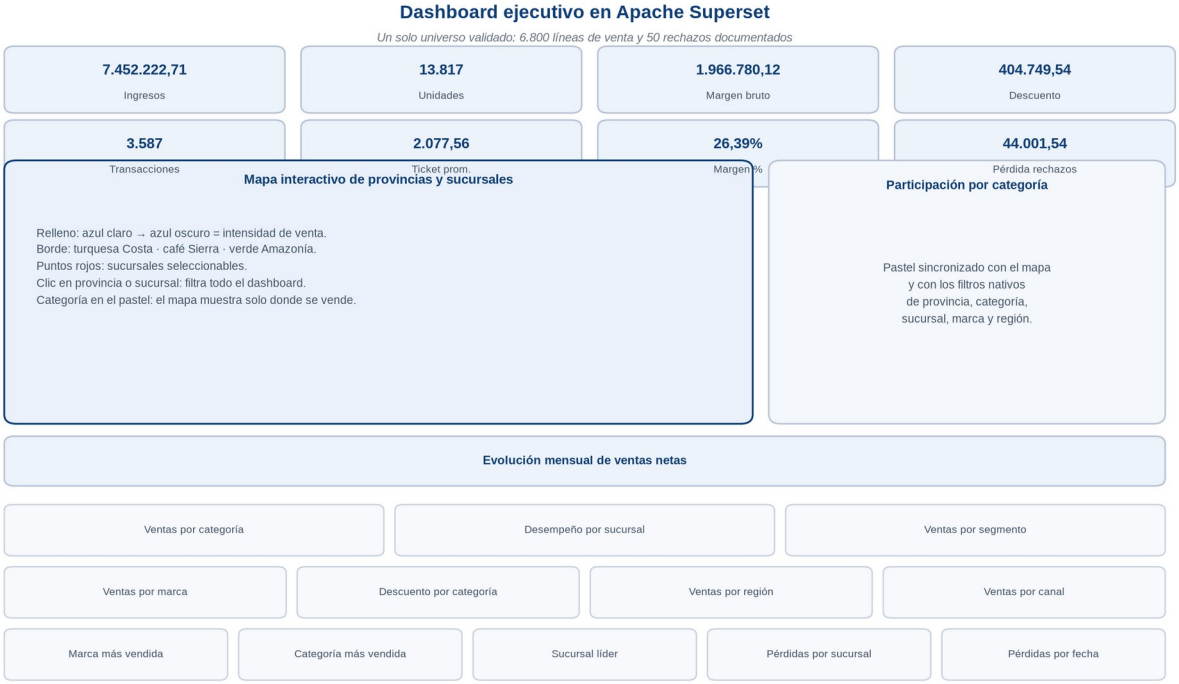
Evidencia del dashboard

La Figura 8 representa la estructura del dashboard ejecutivo publicado en Apache Superset. La primera fila contiene ocho tarjetas de KPI; luego se ubican el mapa provincial y el pastel de participación por categoría; después, la evolución mensual y los cortes de categoría, sucursal, segmento, marca, descuento, región y canal. La sección final muestra los líderes por marca, categoría y sucursal, y las pérdidas por sucursal y fecha.

El mapa funciona como filtro principal. Los polígonos representan provincias y los puntos rojos, sucursales. Al hacer clic en una provincia o en una sucursal, Superset emite un filtro cruzado y todos los gráficos se recalculan. Al seleccionar una categoría en el pastel, el mapa se filtra y colorea por la intensidad de venta de esa categoría, lo que permite identificar las sucursales donde se comercializa. La leyenda del mapa explica la escala de color y el borde regional: turquesa para la Costa, café para la Sierra y verde para la Amazonía. Los filtros nativos de provincia, categoría, sucursal, marca y región complementan la interacción.

Figura 8

Dashboard Sales Analytics generado desde el data warehouse



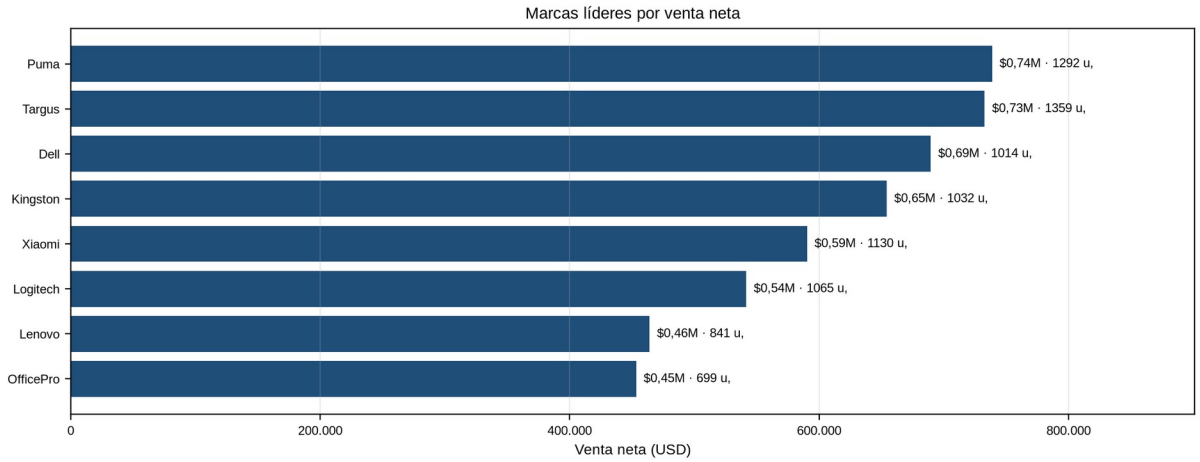
Nota. Esquema del dashboard Sales Analytics en Apache Superset sobre PostgreSQL. La interactividad se verificó con los filtros cruzados y las consultas de control.

Indicadores complementarios de gestión

El descuento otorgado asciende a 404.749,54 USD y se concentra en Accesorios y Hogar. La pérdida estimada por rechazos es 44.001,54 USD sobre 50 registros; la mayor pérdida se observa en Ambato, Santo Domingo y Manta. Estos indicadores permiten priorizar la revisión de descuentos por categoría y la corrección del origen de los registros rechazados, en lugar de tratarlos como una simple exclusión.

Figura 9

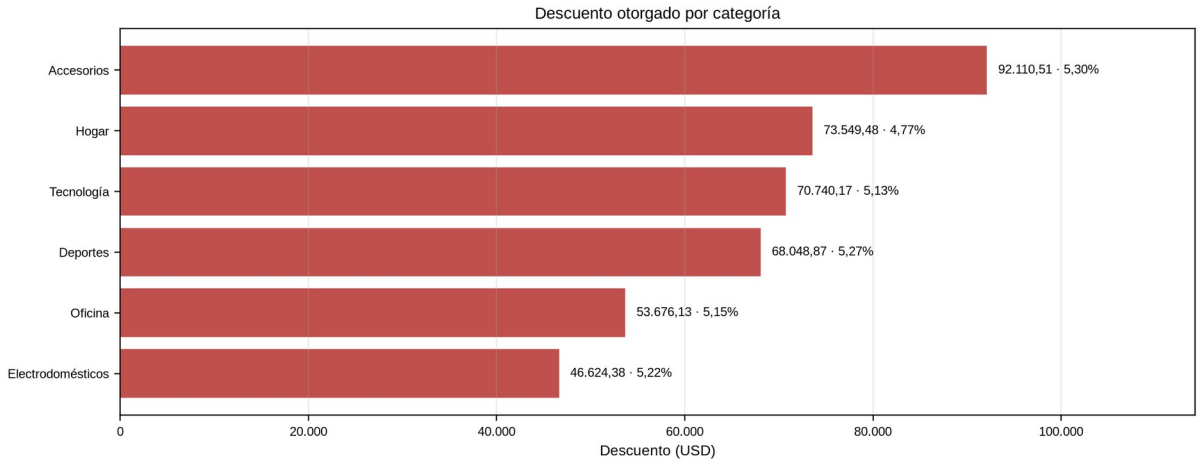
Marcas líderes por venta neta



Nota. Elaboración propia a partir de v_ventas_detalle y DimProducto en 03_dw/dw.db. Puma lidera la venta neta.

Figura 10

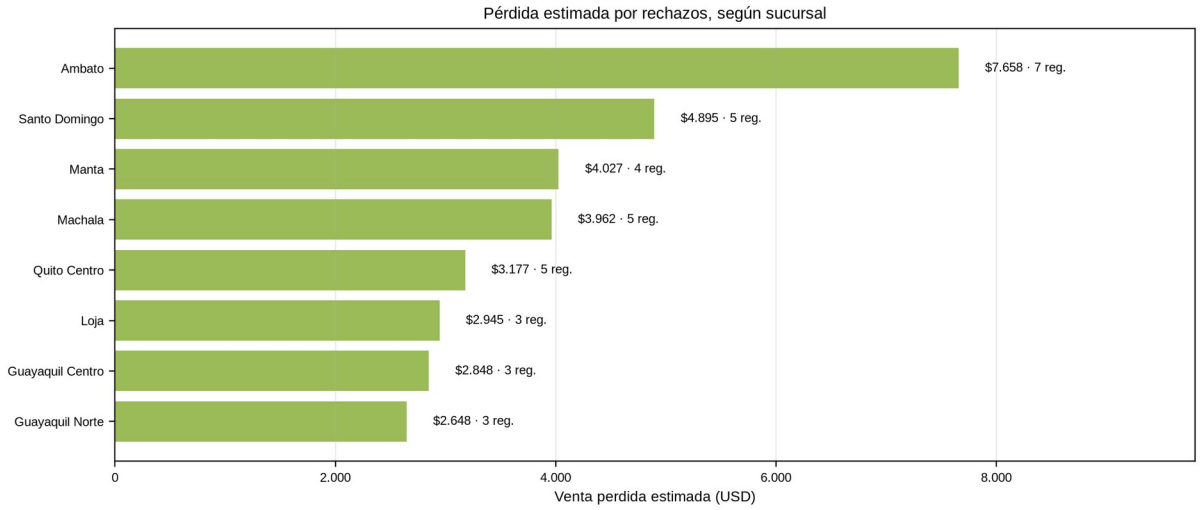
Descuento otorgado por categoría



Nota. Elaboración propia a partir de FactVentas y DimProducto. El descuento se calcula como venta_bruta − venta_neta.

Figura 11

Pérdida estimada por rechazos según sucursal



Nota. Elaboración propia a partir de 06_calidad/log_rechazos.csv. La pérdida se estima como cantidad × precio × (1 − descuento).

Cumplimiento de objetivos

La Tabla 13 relaciona cada objetivo específico con la evidencia producida y su estado. Todos los objetivos se cumplen dentro del alcance de la PoC. La evidencia incluye scripts, reportes de calidad, modelo estrella, vistas KPI, notebook, dashboard y guía de despliegue.

Tabla 13

Cumplimiento de los objetivos específicos

Objetivo	Evidencia	Estado
Integrar las cuatro fuentes	Pipeline 02_etl/main.py y staging limpio	Cumplido
Aplicar reglas de calidad y documentar rechazos	profile_report.md, quality_comparison.md y log_rechazos.csv	Cumplido
Construir el modelo estrella	schema.sql, schema_sqlite.sql y dw.db	Cumplido
Calcular los indicadores comerciales	Consulta de control y notebook 01_etl_pipeline.ipynb	Cumplido
Publicar dashboard interactivo	Apache Superset y dashboard.html	Cumplido
Verificar consistencia y documentar limitaciones	Auditoría de cifras y sección de discusión	Cumplido

Nota. Elaboración propia a partir de los entregables del repositorio.

Auditoría de consistencia de cifras

La auditoría comparó los totales del dashboard con la consulta de control sobre FactVentas. Los ingresos netos coinciden en 7.452.222,71 USD, las unidades en 13.817, el

margen en 1.966.780,12 USD y las transacciones en 3.587. También se verificó que la suma de las categorías, sucursales y segmentos coincida con el total, salvo diferencias de redondeo de centavos.

Durante la auditoría se detectó una discrepancia menor en el reporte de comparación: una fila indicaba 21 fechas inválidas, mientras que el log y la verificación directa mostraban 20. La diferencia se explica porque el reporte contabilizó una fecha inválida que también estaba afectada por otro control y que, tras la depuración, quedó registrada una sola vez. Para el informe se adopta el valor verificable en el log y en la base final. Esta incidencia se documenta como una lección de calidad del propio reporte: los controles también deben auditarse.

Discusión de resultados

Interpretación comercial

El primer hallazgo es que la calidad cambia el universo analítico. De 6.870 líneas de venta, 70 no llegaron al hecho, lo que representa el 1,02 % del total: 20 duplicados y 50 rechazos. Esa proporción puede parecer pequeña, pero su efecto no es neutro: las filas excluidas incluían ventas sin cliente, producto o sucursal, y fechas inválidas que habrían impedido agrupar correctamente la serie temporal. Al excluirlas, el dashboard deja de publicar ingresos que no pueden atribuirse a una dimensión válida.

El segundo hallazgo es la concentración de las ventas. Accesorios y Hogar aportan el 40,39 % de los ingresos, mientras que el segmento Masivo concentra el 50,85 %. Esa estructura sugiere dos líneas de acción: proteger el surtido de las categorías líderes y diseñar campañas de recurrencia para el segmento Frecuente, que aporta 28,45 % y tiene un ticket promedio ligeramente superior al de Masivo. Las sucursales de la Sierra concentran el 57,99 % de los

ingresos, pero la Costa muestra un desempeño cercano, lo que permite comparar prácticas comerciales entre regiones.

El tercer hallazgo es la utilidad del margen como contrapeso a los ingresos. Hogar y Accesorios tienen márgenes brutos similares, aunque Accesorios vende más. Si la gerencia priorizara solo la participación en ventas, podría asignar recursos a una categoría con menor retorno relativo. El tablero, al incluir margen y ticket promedio, permite una lectura más equilibrada y evita decisiones basadas en un único indicador.

Finalmente, la distribución de canales y métodos de pago es equilibrada. Ningún canal supera el 25,44 % ni ningún método supera el 21,11 %. Esa diversificación reduce la dependencia de un único punto de contacto, pero también exige monitorear la calidad de cada fuente: un error en un canal pequeño puede pasar desapercibido, mientras que un error en un canal grande afecta de inmediato los totales.

Contraste con la teoría

Los resultados confirman lo planteado por Kimball y Ross (2013): un modelo dimensional bien definido simplifica las consultas y hace explícito el grano del negocio. En esta PoC, la diferencia entre línea y transacción fue decisiva para calcular correctamente el ticket promedio y el número de operaciones. Si el modelo hubiera mezclado ambos niveles, los indicadores habrían sido inconsistentes aunque las cifras parecieran razonables.

La experiencia también coincide con Batini et al. (2009) y Pipino et al. (2002) en que la calidad debe medirse con dimensiones explícitas. La PoC no se limita a decir que los datos están limpios: cuantifica duplicados, nulos, rechazos y cobertura. Esa evidencia permite comparar el antes y el después, y sostener que el universo final es más confiable aunque sea más pequeño.

El dashboard sigue los principios de Few (2006): prioriza pocos indicadores, evita el exceso de gráficos y ofrece filtros para explorar. La diferencia respecto de un reporte estático es que el usuario puede preguntar por sucursal, categoría o segmento sin solicitar una nueva extracción. La limitación es que la PoC no incluye alertas ni metas, que serían el siguiente nivel de madurez del tablero.

Limitaciones

La primera limitación es el carácter académico y sintético de los datos. Aunque los volúmenes y problemas son realistas, no provienen de una operación comercial en producción. Por tanto, los hallazgos no deben generalizarse a una empresa específica sin una validación con datos reales. La segunda limitación es que la carga es batch y no incremental: el pipeline reconstruye el data warehouse en cada ejecución, lo que es adecuado para una PoC pero no para un volumen alto ni para datos que cambian durante el día.

La tercera limitación es la ausencia de inferencia estadística y de modelos predictivos. El trabajo describe el universo válido y sugiere relaciones, pero no prueba causalidad ni proyecta ventas futuras. La cuarta es que el análisis de costos y rentabilidad se limita al margen bruto: no incluye gastos operativos, logísticos ni de marketing. La quinta es que el gobierno de datos es incipiente: no hay catálogo de datos, dueños de dominio ni acuerdos de nivel de servicio formales.

También existe una limitación técnica: la versión HTML del dashboard es estática y no reemplaza la interactividad de Superset. Se incluyó como evidencia y como plan de contingencia, no como el producto final. En un entorno real, Superset debería ejecutarse con una base de metadatos persistente, respaldo y control de acceso por roles.

Validez y amenazas

La validez interna se apoya en la reproducibilidad del pipeline y en la comparación de totales entre la fuente, el data warehouse y el dashboard. La validez de constructo se sostiene en la definición explícita de cada KPI y en el uso del grano de línea para las métricas. La validez externa es limitada por el carácter académico de los datos y por el periodo de 32 meses; los resultados describen ese caso, no una población de empresas.

La principal amenaza a la conclusión es la tasa de rechazo. Si los registros excluidos se concentraran en una categoría o sucursal, las comparaciones podrían estar sesgadas. Para mitigarlo, el log permite desagregar los rechazos y monitorearlos. Otra amenaza es la calidad del propio reporte de comparación, como mostró la discrepancia de 21 frente a 20 fechas inválidas; por eso se recomienda auditar también los reportes de control y no solo los datos de origen.

Conclusiones

La prueba de concepto demostró que es posible integrar cuatro fuentes comerciales heterogéneas y producir indicadores trazables con herramientas abiertas. El pipeline ETL documenta cada transformación, el modelo estrella separa hechos y dimensiones, y el dashboard presenta los resultados sobre el mismo universo validado.

La calidad de datos no es un paso accesorio: modificó el universo de 6.870 a 6.800 líneas y evitó que ventas huérfanas o duplicadas distorsionaran los totales. La cobertura final de 98,98 % es alta, pero debe monitorearse porque una tasa de rechazo creciente reduciría la representatividad del análisis.

Los indicadores consolidados son consistentes: 7.452.222,71 USD de ingresos netos, 13.817 unidades, 1.966.780,12 USD de margen bruto y 3.587 transacciones. El margen

representa 26,39% de los ingresos, un dato que debe leerse junto con la categoría y la sucursal, no de forma aislada.

El dashboard permite identificar que Accesorios y Hogar concentran la mayor participación, que el segmento Masivo explica la mitad de los ingresos y que la Sierra aporta cerca del 58 % de las ventas. Esas lecturas son un punto de partida para decisiones de inventario, campañas y asignación comercial, no una respuesta definitiva.

La principal contribución metodológica es haber convertido la calidad en evidencia cuantificable. El log de rechazos, la comparación antes/después y la auditoría de totales permiten defender cada cifra ante un tribunal o una mesa de decisiones. Esa trazabilidad es más valiosa que la cifra aislada.

Como trabajo futuro se recomienda incorporar carga incremental, orquestación, pruebas automatizadas, catálogo de datos y metas por sucursal. Con esas mejoras, la PoC podría evolucionar hacia un producto de BI operativo sin perder la trazabilidad que la caracteriza.

Recomendaciones

1. Implementar carga incremental y un orquestador (por ejemplo, Airflow o un job programado) para evitar reconstruir todo el data warehouse en cada ejecución.
2. Automatizar pruebas de calidad y de regresión sobre los totales, de modo que un cambio en el origen no pase inadvertido.
3. Monitorear la tasa de rechazo por fuente, sucursal y categoría, y establecer un umbral de alerta cuando supere el nivel aceptado por el negocio.
4. Documentar un catálogo de datos con definiciones de KPIs, dueños de dominio, granularidad y reglas de negocio.

5. Incluir metas y presupuestos por sucursal para que el dashboard no solo describa, sino que también evalúe cumplimiento.

6. Ampliar el análisis de rentabilidad con costos operativos y logísticos, porque el margen bruto no refleja la utilidad final.

7. Reforzar la seguridad de Superset con roles, autenticación corporativa y respaldo de la base de metadatos.

8. Validar la solución con datos reales anonimizados antes de generalizar los hallazgos a una organización específica.

Referencias

Apache Software Foundation. (2026). *Apache Superset documentation*. https://superset.apache.org/docs/intro

Batini, C., Cappiello, C., Francalanci, C., & Maurino, A. (2009). Methodologies for data quality assessment and improvement. *ACM Computing Surveys, 41*(3), 1–52. https://doi.org/10.1145/1541880.1541883

Docker Inc. (2026). *Docker documentation*. https://docs.docker.com/

Few, S. (2006). *Information dashboard design: The effective visual communication of data*. O'Reilly Media.

Hernández-Sampieri, R., & Mendoza, C. (2018). *Metodología de la investigación: Las rutas cuantitativa, cualitativa y mixta*. McGraw-Hill.

ISO. (2016). *ISO 8000-61:2016 Data quality — Part 61: Data quality management: Process reference model*. International Organization for Standardization. https://www.iso.org/standard/62383.html

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd ed.). Wiley.

Northflank. (2026). *Northflank documentation*. https://northflank.com/docs

pandas Development Team. (2026). *pandas documentation*. https://pandas.pydata.org/docs/

Pipino, L. L., Lee, Y. W., & Wang, R. Y. (2002). Data quality assessment. *Communications of the ACM, 45*(4), 211–218. https://doi.org/10.1145/505999.506010

PostgreSQL Global Development Group. (2026). *PostgreSQL documentation*. https://www.postgresql.org/docs/

Provost, F., & Fawcett, T. (2013). *Data science for business: What you need to know about data mining and data-analytic thinking*. O'Reilly Media.

Sharda, R., Delen, D., & Turban, E. (2020). *Analytics, data science, & artificial intelligence: Systems for decision support* (11th ed.). Pearson.

Universidad Central del Ecuador. (2022). *Lineamientos para la publicación de trabajos de titulación de pregrado y posgrado con aplicación de la norma APA 7 en el repositorio institucional*. UCE.

Universidad Central del Ecuador, Facultad de Ingeniería y Ciencias Aplicadas. (2026). *Guía metodológica del examen complexivo 2025-2026*. UCE.

Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. *Proceedings of the 4th International Conference on the Practical Application of Knowledge Discovery and Data Mining*, 29–39.

Anexos

Anexo A. Esquema del modelo estrella

El siguiente extracto corresponde al DDL PostgreSQL utilizado para crear el modelo estrella. El esquema SQLite mantiene la misma estructura y se usa como respaldo local para la evidencia.

```
-- Star Schema adaptado a PostgreSQL
-- Data Warehouse para PoC Integración y Calidad de Datos

-- DIMENSIONES
CREATE TABLE IF NOT EXISTS dim_tiempo (
    id_tiempo INTEGER PRIMARY KEY,
    fecha DATE NOT NULL,
    anio INTEGER,
    mes INTEGER,
    mes_nombre VARCHAR(20),
    trimestre VARCHAR(2),
    semana INTEGER,
    dia_semana VARCHAR(15)
);

CREATE TABLE IF NOT EXISTS dim_cliente (
    id_cliente SERIAL PRIMARY KEY,
    cod_cliente VARCHAR(10) UNIQUE NOT NULL,
    nombre_cliente VARCHAR(150),
    segmento VARCHAR(30),
    ciudad VARCHAR(80),
    provincia VARCHAR(80),
    fecha_registro DATE,
    estado_cliente VARCHAR(15)
);

CREATE TABLE IF NOT EXISTS dim_producto (
    id_producto SERIAL PRIMARY KEY,
    cod_producto VARCHAR(10) UNIQUE NOT NULL,
    nombre_producto VARCHAR(150),
    categoria VARCHAR(60),
    subcategoria VARCHAR(60),
    marca VARCHAR(60),
    costo_unitario NUMERIC(18,6),
    estado_producto VARCHAR(15)
);

CREATE TABLE IF NOT EXISTS dim_sucursal (
    id_sucursal SERIAL PRIMARY KEY,
    cod_sucursal VARCHAR(10) UNIQUE NOT NULL,
    nombre_sucursal VARCHAR(80),
    ciudad VARCHAR(80),
    provincia VARCHAR(80),
    region VARCHAR(20),
```

```
    fecha_apertura DATE,
    estado_sucursal VARCHAR(15)
);

-- HECHO
CREATE TABLE IF NOT EXISTS fact_ventas (
    id_venta_pk SERIAL PRIMARY KEY,
    id_venta VARCHAR(15) NOT NULL,
    id_tiempo INTEGER REFERENCES dim_tiempo(id_tiempo),
    id_cliente INTEGER REFERENCES dim_cliente(id_cliente),
    id_producto INTEGER REFERENCES dim_producto(id_producto),
    id_sucursal INTEGER REFERENCES dim_sucursal(id_sucursal),
    cantidad INTEGER,
    precio_unitario NUMERIC(18,6),
    descuento_pct NUMERIC(8,6),
    venta_bruta NUMERIC(18,6),
    venta_neta NUMERIC(18,6),
    costo_total NUMERIC(18,6),
    margen NUMERIC(18,6),
    canal_venta VARCHAR(30),
    metodo_pago VARCHAR(30)
);

-- HECHO DE RECHAZOS: registros que no pudieron integrarse al hecho
principal.
-- Se conservan como perdida estimada y trazabilidad de calidad.
CREATE TABLE IF NOT EXISTS fact_rechazos (
    id_rechazo SERIAL PRIMARY KEY,
    id_venta VARCHAR(15),
    fecha_venta DATE,
    cod_cliente VARCHAR(15),
    cod_producto VARCHAR(15),
    cod_sucursal VARCHAR(15),
    cantidad NUMERIC(18,6),
    precio_unitario NUMERIC(18,6),
    descuento_pct NUMERIC(8,6),
    canal_venta VARCHAR(30),
    metodo_pago VARCHAR(30),
    motivo VARCHAR(200),
    venta_perdida NUMERIC(18,6)
);

-- ÍNDICES PARA RENDIMIENTO
CREATE INDEX IF NOT EXISTS idx_fact_ventas_tiempo ON
fact_ventas(id_tiempo);
CREATE INDEX IF NOT EXISTS idx_fact_ventas_cliente ON
fact_ventas(id_cliente);
CREATE INDEX IF NOT EXISTS idx_fact_ventas_producto ON
fact_ventas(id_producto);
CREATE INDEX IF NOT EXISTS idx_fact_ventas_sucursal ON
fact_ventas(id_sucursal);
CREATE INDEX IF NOT EXISTS idx_fact_ventas_idventa ON
fact_ventas(id_venta);

-- Mapa provincial para el dashboard de Superset
CREATE TABLE IF NOT EXISTS dim_provincia_mapa (
```

```
    provincia VARCHAR(80) PRIMARY KEY,
    polygon TEXT NOT NULL,
    lat NUMERIC(10,6),
    lon NUMERIC(10,6)
);
```

Anexo B. Vistas KPI

Las vistas KPI encapsulan los agregados que consume Apache Superset. Se recrean en cada carga para garantizar que el dashboard y la evidencia utilicen el mismo universo validado.

```
-- Vistas KPI para PostgreSQL / Apache Superset
-- Se recrean en cada carga del ETL.
-- Modelo analitico: un hecho de ventas validas (fact_ventas) y un hecho de
-- rechazos (fact_rechazos) para medir perdidas por sucursal y fecha.

-- ======================================================================
-- VISTA UNIFICADA A NIVEL DE DETALLE (habilita cross-filtering completo)
-- ======================================================================
DROP VIEW IF EXISTS v_ventas_detalle CASCADE;
CREATE VIEW v_ventas_detalle AS
SELECT f.id_venta_pk,
       f.id_venta,
       t.fecha,
       t.anio,
       t.mes,
       t.mes_nombre,
       t.trimestre,
       t.semana,
       t.dia_semana,
       c.provincia,
       c.ciudad AS ciudad_cliente,
       c.segmento,
       p.cod_producto,
       p.nombre_producto,
       p.categoria,
       p.subcategoria,
       p.marca,
       s.id_sucursal,
       s.nombre_sucursal,
       s.ciudad AS ciudad_sucursal,
       s.provincia AS provincia_sucursal,
       s.region,
       f.canal_venta,
       f.metodo_pago,
       f.cantidad,
       f.precio_unitario,
       f.descuento_pct,
       f.venta_bruta,
       (f.venta_bruta - f.venta_neta) AS descuento_usd,
       f.venta_neta,
       f.costo_total,
       f.margen
```

```
FROM fact_ventas f
JOIN dim_tiempo t ON f.id_tiempo = t.id_tiempo
JOIN dim_cliente c ON f.id_cliente = c.id_cliente
JOIN dim_producto p ON f.id_producto = p.id_producto
JOIN dim_sucursal s ON f.id_sucursal = s.id_sucursal;

-- ========================================================================
-- VISTAS DE VENTAS AGREGADAS
-- ========================================================================
DROP VIEW IF EXISTS v_ventas_mes CASCADE;
CREATE VIEW v_ventas_mes AS
SELECT anio, mes, mes_nombre, trimestre,
       make_date(anio, mes, 1) AS fecha,
       SUM(venta_neta) AS venta_neta,
       SUM(cantidad) AS unidades,
       COUNT(DISTINCT id_venta) AS num_ventas
FROM v_ventas_detalle
GROUP BY anio, mes, mes_nombre, trimestre;

DROP VIEW IF EXISTS v_ventas_categoria CASCADE;
CREATE VIEW v_ventas_categoria AS
SELECT categoria,
       SUM(venta_neta) AS venta_neta,
       SUM(cantidad) AS unidades,
       SUM(descuento_usd) AS descuento_usd,
       SUM(margen) AS margen,
       COUNT(DISTINCT id_venta) AS num_ventas,
       SUM(venta_neta) * 100.0 / NULLIF((SELECT SUM(venta_neta) FROM
fact_ventas), 0) AS pct_participacion
FROM v_ventas_detalle
GROUP BY categoria;

DROP VIEW IF EXISTS v_ventas_marca CASCADE;
CREATE VIEW v_ventas_marca AS
SELECT marca,
       SUM(cantidad) AS unidades,
       SUM(venta_neta) AS venta_neta,
       SUM(descuento_usd) AS descuento_usd,
       SUM(margen) AS margen,
       COUNT(DISTINCT id_venta) AS num_ventas,
       AVG(descuento_pct) AS descuento_promedio
FROM v_ventas_detalle
WHERE marca IS NOT NULL AND TRIM(marca) <> ''
GROUP BY marca;

DROP VIEW IF EXISTS v_ventas_sucursal CASCADE;
CREATE VIEW v_ventas_sucursal AS
SELECT id_sucursal,
       nombre_sucursal,
       ciudad_sucursal,
       provincia_sucursal,
       region,
       SUM(venta_neta) AS venta_neta,
       SUM(cantidad) AS unidades,
       SUM(descuento_usd) AS descuento_usd,
       SUM(margen) AS margen,
```

```
       COUNT(DISTINCT id_venta) AS num_ventas,
       SUM(venta_neta) / NULLIF(COUNT(DISTINCT id_venta), 0) AS
ticket_promedio
FROM v_ventas_detalle
GROUP BY id_sucursal, nombre_sucursal, ciudad_sucursal, provincia_sucursal,
region;

DROP VIEW IF EXISTS v_ventas_segmento CASCADE;
CREATE VIEW v_ventas_segmento AS
SELECT segmento,
       SUM(venta_neta) AS venta_neta,
       SUM(cantidad) AS unidades,
       SUM(descuento_usd) AS descuento_usd,
       SUM(margen) AS margen,
       COUNT(DISTINCT id_venta) AS num_ventas,
       SUM(venta_neta) / NULLIF(COUNT(DISTINCT id_venta), 0) AS
ticket_promedio
FROM v_ventas_detalle
GROUP BY segmento;

DROP VIEW IF EXISTS v_ventas_canal CASCADE;
CREATE VIEW v_ventas_canal AS
SELECT canal_venta,
       SUM(venta_neta) AS venta_neta,
       SUM(cantidad) AS unidades,
       COUNT(*) AS lineas
FROM v_ventas_detalle
GROUP BY canal_venta;

-- ======================================================================
-- MATRICES DE DECISION: categoria y sucursal dentro de cada provincia
-- ======================================================================
DROP VIEW IF EXISTS v_categoria_provincia CASCADE;
CREATE VIEW v_categoria_provincia AS
SELECT provincia,
       categoria,
       SUM(venta_neta) AS venta_neta,
       SUM(cantidad) AS unidades,
       SUM(descuento_usd) AS descuento_usd,
       SUM(margen) AS margen,
       COUNT(DISTINCT id_venta) AS num_ventas
FROM v_ventas_detalle
GROUP BY provincia, categoria;

DROP VIEW IF EXISTS v_sucursal_provincia CASCADE;
CREATE VIEW v_sucursal_provincia AS
SELECT provincia_sucursal AS provincia,
       nombre_sucursal,
       SUM(venta_neta) AS venta_neta,
       SUM(cantidad) AS unidades,
       SUM(descuento_usd) AS descuento_usd,
       SUM(margen) AS margen,
       COUNT(DISTINCT id_venta) AS num_ventas,
       SUM(venta_neta) / NULLIF(COUNT(DISTINCT id_venta), 0) AS
ticket_promedio
FROM v_ventas_detalle
```

```
GROUP BY provincia_sucursal, nombre_sucursal;

DROP VIEW IF EXISTS v_descuento_categoria_provincia CASCADE;
CREATE VIEW v_descuento_categoria_provincia AS
SELECT provincia,
       categoria,
       SUM(venta_bruta) AS venta_bruta,
       SUM(venta_neta) AS venta_neta,
       SUM(descuento_usd) AS descuento_usd,
       AVG(descuento_pct) AS descuento_promedio,
       COUNT(DISTINCT id_venta) AS num_ventas
FROM v_ventas_detalle
GROUP BY provincia, categoria;

DROP VIEW IF EXISTS v_descuento_sucursal CASCADE;
CREATE VIEW v_descuento_sucursal AS
SELECT provincia_sucursal AS provincia,
       nombre_sucursal,
       SUM(venta_bruta) AS venta_bruta,
       SUM(venta_neta) AS venta_neta,
       SUM(descuento_usd) AS descuento_usd,
       AVG(descuento_pct) AS descuento_promedio,
       COUNT(DISTINCT id_venta) AS num_ventas
FROM v_ventas_detalle
GROUP BY provincia_sucursal, nombre_sucursal;

-- ========================================================================
-- RANKINGS PARA DECISION
-- ========================================================================
DROP VIEW IF EXISTS v_top_marca_provincia CASCADE;
CREATE VIEW v_top_marca_provincia AS
WITH ranking AS (
    SELECT provincia, marca,
           SUM(cantidad) AS unidades,
           SUM(venta_neta) AS venta_neta,
           SUM(descuento_usd) AS descuento_usd,
           ROW_NUMBER() OVER (
               PARTITION BY provincia
               ORDER BY SUM(cantidad) DESC, SUM(venta_neta) DESC
           ) AS rn
    FROM v_ventas_detalle
    WHERE marca IS NOT NULL AND TRIM(marca) <> ''
    GROUP BY provincia, marca
)
SELECT provincia, marca, unidades, venta_neta, descuento_usd
FROM ranking WHERE rn = 1;

DROP VIEW IF EXISTS v_top_categoria_sucursal CASCADE;
CREATE VIEW v_top_categoria_sucursal AS
WITH ranking AS (
    SELECT nombre_sucursal, provincia_sucursal AS provincia, categoria,
           SUM(cantidad) AS unidades,
           SUM(venta_neta) AS venta_neta,
           ROW_NUMBER() OVER (
               PARTITION BY nombre_sucursal
               ORDER BY SUM(cantidad) DESC, SUM(venta_neta) DESC
```

```
            ) AS rn
    FROM v_ventas_detalle
    GROUP BY nombre_sucursal, provincia_sucursal, categoria
)
SELECT nombre_sucursal, provincia, categoria, unidades, venta_neta
FROM ranking WHERE rn = 1;

DROP VIEW IF EXISTS v_top_producto_sucursal CASCADE;
CREATE VIEW v_top_producto_sucursal AS
WITH ranking AS (
    SELECT nombre_sucursal, provincia_sucursal AS provincia,
           cod_producto, nombre_producto, categoria, marca,
           SUM(cantidad) AS unidades,
           SUM(venta_neta) AS venta_neta,
           ROW_NUMBER() OVER (
               PARTITION BY nombre_sucursal
               ORDER BY SUM(cantidad) DESC, SUM(venta_neta) DESC
           ) AS rn
    FROM v_ventas_detalle
    GROUP BY nombre_sucursal, provincia_sucursal, cod_producto,
             nombre_producto, categoria, marca
)
SELECT nombre_sucursal, provincia, cod_producto, nombre_producto,
       categoria, marca, unidades, venta_neta
FROM ranking WHERE rn = 1;

-- ======================================================================
-- MAPA COROPLETICO PROVINCIAL (poligono en JSON para deck.gl)
-- ======================================================================
DROP VIEW IF EXISTS v_ventas_provincia CASCADE;
CREATE VIEW v_ventas_provincia AS
SELECT m.provincia,
       m.polygon AS geom,
       m.lat,
       m.lon,
       COALESCE(SUM(f.venta_neta), 0) AS venta_neta,
       COALESCE(SUM(f.cantidad), 0) AS unidades,
       COALESCE(SUM(f.margen), 0) AS margen,
       COALESCE(SUM(f.venta_bruta - f.venta_neta), 0) AS descuento_usd,
       COUNT(DISTINCT f.id_venta) AS num_ventas
FROM dim_provincia_mapa m
LEFT JOIN dim_cliente c ON c.provincia = m.provincia
LEFT JOIN fact_ventas f ON f.id_cliente = c.id_cliente
GROUP BY m.provincia, m.polygon, m.lat, m.lon;

DROP VIEW IF EXISTS v_sucursales_mapa CASCADE;
CREATE VIEW v_sucursales_mapa AS
SELECT s.id_sucursal,
       s.nombre_sucursal,
       s.ciudad,
       s.provincia,
       s.region,
       m.lat,
       m.lon,
       m.polygon AS geom,
       COALESCE(SUM(f.venta_neta), 0) AS venta_neta,
```

```
       COALESCE(SUM(f.venta_bruta - f.venta_neta), 0) AS descuento_usd,
       COUNT(DISTINCT f.id_venta) AS num_ventas
FROM dim_sucursal s
LEFT JOIN dim_provincia_mapa m ON m.provincia = s.provincia
LEFT JOIN fact_ventas f ON f.id_sucursal = s.id_sucursal
GROUP BY s.id_sucursal, s.nombre_sucursal, s.ciudad, s.provincia, s.region,
         m.lat, m.lon, m.polygon;

-- Mapa unico: provincias (poligonos) y sucursales (puntos).
-- Se replica una fila por categoria para que el mapa reaccione al filtro
de
-- categoria; el color se calcula sobre el total provincial.
DROP VIEW IF EXISTS v_mapa_consolidado CASCADE;
CREATE VIEW v_mapa_consolidado AS
WITH prov AS (
    SELECT m.provincia,
           m.polygon AS geom,
           COALESCE(SUM(f.venta_neta), 0) AS venta_neta,
           COUNT(DISTINCT f.id_venta) AS num_ventas
    FROM dim_provincia_mapa m
    LEFT JOIN dim_cliente c ON c.provincia = m.provincia
    LEFT JOIN fact_ventas f ON f.id_cliente = c.id_cliente
    GROUP BY m.provincia, m.polygon
    HAVING COALESCE(SUM(f.venta_neta), 0) > 0
),
prov_color AS (
    SELECT *, NTILE(5) OVER (ORDER BY venta_neta) AS bucket FROM prov
),
prov_region AS (
    SELECT * FROM (VALUES
        ('Esmeraldas', 'Costa'),
        ('Manabí', 'Costa'),
        ('Santo Domingo de los Tsáchilas', 'Costa'),
        ('Los Ríos', 'Costa'),
        ('Guayas', 'Costa'),
        ('Santa Elena', 'Costa'),
        ('El Oro', 'Costa'),
        ('Carchi', 'Sierra'),
        ('Imbabura', 'Sierra'),
        ('Pichincha', 'Sierra'),
        ('Cotopaxi', 'Sierra'),
        ('Tungurahua', 'Sierra'),
        ('Bolívar', 'Sierra'),
        ('Chimborazo', 'Sierra'),
        ('Cañar', 'Sierra'),
        ('Azuay', 'Sierra'),
        ('Loja', 'Sierra'),
        ('Sucumbíos', 'Amazonía'),
        ('Napo', 'Amazonía'),
        ('Orellana', 'Amazonía'),
        ('Pastaza', 'Amazonía'),
        ('Morona Santiago', 'Amazonía'),
        ('Zamora Chinchipe', 'Amazonía')
    ) AS t(provincia, region)
),
prov_cat_sales AS (
```

```
    SELECT c.provincia,
           p.categoria,
           SUM(f.venta_neta) AS venta_cat,
           COUNT(DISTINCT f.id_venta) AS num_ventas_cat
    FROM fact_ventas f
    JOIN dim_cliente c ON f.id_cliente = c.id_cliente
    JOIN dim_producto p ON f.id_producto = p.id_producto
    GROUP BY c.provincia, p.categoria
),
prov_features AS (
    SELECT pcs.provincia,
           pcs.categoria,
           NULL::text AS nombre_sucursal,
           COALESCE(pr.region, 'Sin región') AS region,
           ('provincia:' || pcs.provincia) AS geo_key,
           jsonb_build_object(
               'type', 'Feature',
               'provincia', pcs.provincia,
               'geo_key', 'provincia:' || pcs.provincia,
               'geometry', jsonb_build_object(
                   'type', 'Polygon',
                   'coordinates', jsonb_build_array(pv.geom::jsonb)
               ),
               'properties', jsonb_build_object(
                   'tipo', 'provincia',
                   'provincia', pcs.provincia,
                   'categoria', pcs.categoria,
                   'region', COALESCE(pr.region, 'Sin región'),
                   'ventaNeta', round(pcs.venta_cat, 2),
                   'ventaTotal', round(pv.venta_neta, 2),
                   'fillColor',
                   (ARRAY['#c9dcff', '#9bbcf7', '#5f8fe8', '#2a5fc4',
'#0033A0'])[pv.bucket],
                   'strokeColor', '#ffffff',
                   'stroke-width', 1.5
               ),
               'extraProps', jsonb_build_object(
                   'Provincia', pcs.provincia,
                   'Categoría', pcs.categoria,
                   'Venta neta', round(pcs.venta_cat, 2),
                   'Venta total', round(pv.venta_neta, 2),
                   'Transacciones', pcs.num_ventas_cat
               )
           )::text AS geojson
    FROM prov_color pv
    JOIN prov_cat_sales pcs ON pcs.provincia = pv.provincia
    LEFT JOIN prov_region pr ON pr.provincia = pcs.provincia
),
suc AS (
    SELECT s.nombre_sucursal,
           s.ciudad,
           s.provincia,
           s.region,
           m.lat,
           m.lon,
           COALESCE(SUM(f.venta_neta), 0) AS venta_neta,
           COUNT(DISTINCT f.id_venta) AS num_ventas
```

```
    FROM dim_sucursal s
    JOIN dim_provincia_mapa m ON m.provincia = s.provincia
    LEFT JOIN fact_ventas f ON f.id_sucursal = s.id_sucursal
    GROUP BY s.nombre_sucursal, s.ciudad, s.provincia, s.region, m.lat,
m.lon
),
suc_pos AS (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY provincia ORDER BY nombre_sucursal
           ) AS rn_prov,
           COUNT(*) OVER (PARTITION BY provincia) AS cnt_prov
    FROM suc
),
suc_cat_sales AS (
    SELECT s.nombre_sucursal,
           p.categoria,
           SUM(f.venta_neta) AS venta_cat,
           COUNT(DISTINCT f.id_venta) AS num_ventas_cat
    FROM fact_ventas f
    JOIN dim_sucursal s ON f.id_sucursal = s.id_sucursal
    JOIN dim_producto p ON f.id_producto = p.id_producto
    GROUP BY s.nombre_sucursal, p.categoria
),
suc_features AS (
    SELECT sp.provincia,
           scs.categoria,
           sp.nombre_sucursal,
           COALESCE(sp.region, 'Sin región') AS region,
           ('sucursal:' || sp.nombre_sucursal) AS geo_key,
           jsonb_build_object(
               'type', 'Feature',
               'provincia', sp.provincia,
               'geo_key', 'sucursal:' || sp.nombre_sucursal,
               'geometry', jsonb_build_object(
                   'type', 'Point',
                   'coordinates', jsonb_build_array(
                       sp.lon + CASE
                           WHEN sp.cnt_prov = 1 THEN 0
                           ELSE (sp.rn_prov - (sp.cnt_prov + 1) / 2.0) *
0.10
                       END,
                       sp.lat + CASE
                           WHEN sp.cnt_prov = 1 THEN 0
                           WHEN MOD(sp.rn_prov, 2) = 0 THEN 0.06
                           ELSE -0.06
                       END
                   )
               ),
               'properties', jsonb_build_object(
                   'tipo', 'sucursal',
                   'sucursal', sp.nombre_sucursal,
                   'ciudad', sp.ciudad,
                   'provincia', sp.provincia,
                   'region', COALESCE(sp.region, 'Sin región'),
                   'categoria', scs.categoria,
                   'ventaNeta', round(scs.venta_cat, 2),
```

```
                    'ventaTotal', round(sp.venta_neta, 2),
                    'fillColor', '#CE1126',
                    'strokeColor', '#ffffff',
                    'stroke-width', 2
                ),
                'extraProps', jsonb_build_object(
                    'Sucursal', sp.nombre_sucursal,
                    'Ciudad', sp.ciudad,
                    'Provincia', sp.provincia,
                    'Categoría', scs.categoria,
                    'Venta neta', round(scs.venta_cat, 2),
                    'Venta total', round(sp.venta_neta, 2),
                    'Transacciones', scs.num_ventas_cat
                )
            )::text AS geojson
    FROM suc_pos sp
    JOIN suc_cat_sales scs ON scs.nombre_sucursal = sp.nombre_sucursal
)
SELECT geojson, provincia, categoria, nombre_sucursal, region, geo_key FROM
prov_features
UNION ALL
SELECT geojson, provincia, categoria, nombre_sucursal, region, geo_key FROM
suc_features;

-- ========================================================================
-- RECHAZOS: perdidas estimadas por sucursal, fecha y motivo
-- ========================================================================
DROP VIEW IF EXISTS v_rechazos CASCADE;
CREATE VIEW v_rechazos AS
SELECT r.id_rechazo,
       r.id_venta,
       r.fecha_venta,
       r.cod_cliente,
       r.cod_producto,
       r.cod_sucursal,
       COALESCE(s.nombre_sucursal, r.cod_sucursal, 'Sucursal desconocida')
AS nombre_sucursal,
       COALESCE(s.provincia, 'Provincia desconocida') AS provincia,
       COALESCE(s.region, 'Sin región') AS region,
       COALESCE(p.categoria, 'Sin categoría') AS categoria,
       COALESCE(p.marca, 'Sin marca') AS marca,
       r.cantidad,
       r.precio_unitario,
       r.descuento_pct,
       r.canal_venta,
       r.metodo_pago,
       COALESCE(r.motivo, 'Sin motivo') AS motivo,
       r.venta_perdida
FROM fact_rechazos r
LEFT JOIN dim_sucursal s ON s.cod_sucursal = r.cod_sucursal
LEFT JOIN dim_producto p ON p.cod_producto = r.cod_producto;

DROP VIEW IF EXISTS v_rechazos_sucursal CASCADE;
CREATE VIEW v_rechazos_sucursal AS
SELECT nombre_sucursal,
       provincia,
```

```
       COUNT(*) AS num_rechazos,
       SUM(venta_perdida) AS venta_perdida
FROM v_rechazos
GROUP BY nombre_sucursal, provincia;

DROP VIEW IF EXISTS v_rechazos_fecha CASCADE;
CREATE VIEW v_rechazos_fecha AS
SELECT fecha_venta,
       date_trunc('month', fecha_venta)::date AS mes,
       COUNT(*) AS num_rechazos,
       SUM(venta_perdida) AS venta_perdida
FROM v_rechazos
WHERE fecha_venta IS NOT NULL
GROUP BY fecha_venta, date_trunc('month', fecha_venta)::date;

DROP VIEW IF EXISTS v_rechazos_motivo CASCADE;
CREATE VIEW v_rechazos_motivo AS
SELECT motivo,
       COUNT(*) AS num_rechazos,
       SUM(venta_perdida) AS venta_perdida
FROM v_rechazos
GROUP BY motivo;
```

Anexo C. Evidencia de rechazos

El archivo log_rechazos.csv contiene 50 registros. A continuación se presenta una muestra de las primeras filas; el archivo completo se conserva en 06_calidad/.

```
id_venta,fecha_venta,cod_cliente,cod_producto,cod_sucursal,cantidad,precio_
unitario,descuento_pct,canal_venta,metodo_pago,motivo
V000665,2024-12-
24,CX9216,P0065,S005,1,230.62,0.1,Marketplace,Transferencia,FK invalida:
cod_cliente no existe en cod_cliente
V000903,2026-07-24,CX9488,P0088,S007,2,577.14,0.0,Marketplace,Tarjeta de
crédito,FK invalida: cod_cliente no existe en cod_cliente
V001262,2025-05-19,CX9158,P0076,S003,3,585.73,0.0,Web,Transferencia,FK
invalida: cod_cliente no existe en cod_cliente
V001368,2024-11-09,CX9606,P0079,S001,1,298.7,0.05,Aplicación móvil,Tarjeta
de crédito,FK invalida: cod_cliente no existe en cod_cliente
V001493,2025-08-18,CX9318,P0034,S001,2,779.85,0.0,Marketplace,Tarjeta de
crédito,FK invalida: cod_cliente no existe en cod_cliente
V001699,2024-03-18,CX9557,P0091,S002,2,829.77,0.0,Aplicación
móvil,Billetera digital,FK invalida: cod_cliente no existe en cod_cliente
V001823,2025-06-16,CX9381,P0051,S012,1,753.48,0.1,Aplicación
móvil,Efectivo,FK invalida: cod_cliente no existe en cod_cliente
V002463,2025-12-28,CX9734,P0031,S001,1,153.59,0.0,Aplicación
móvil,Efectivo,FK invalida: cod_cliente no existe en cod_cliente
V002493,2024-08-18,CX9698,P0076,S008,4,587.5,0.0,Tienda
física,Transferencia,FK invalida: cod_cliente no existe en cod_cliente
V002884,2025-07-29,CX9326,P0038,S005,2,485.36,0.0,Web,Transferencia,FK
invalida: cod_cliente no existe en cod_cliente
```

Anexo D. Reproducibilidad

Los comandos siguientes permiten regenerar la evidencia y el dashboard desde la raíz del repositorio (https://github.com/DavCoder22/Proyecto34). El ETL utiliza SQLite en local y PostgreSQL o Neon cuando ETL_TARGET=postgres.

```
python 02_etl/main.py
python 04_dashboard/build_dashboard.py

docker compose -f docker/docker-compose.yml up --build
docker compose -f docker/docker-compose.superset.yml up -d --build
docker compose -f docker/docker-compose.superset.yml run --rm etl

# Archivos de evidencia
06_calidad/profile_report.md
06_calidad/quality_comparison.md
06_calidad/reglas_transformacion.md
06_calidad/log_rechazos.csv
03_dw/dw.db
02_etl/01_etl_pipeline.ipynb
```

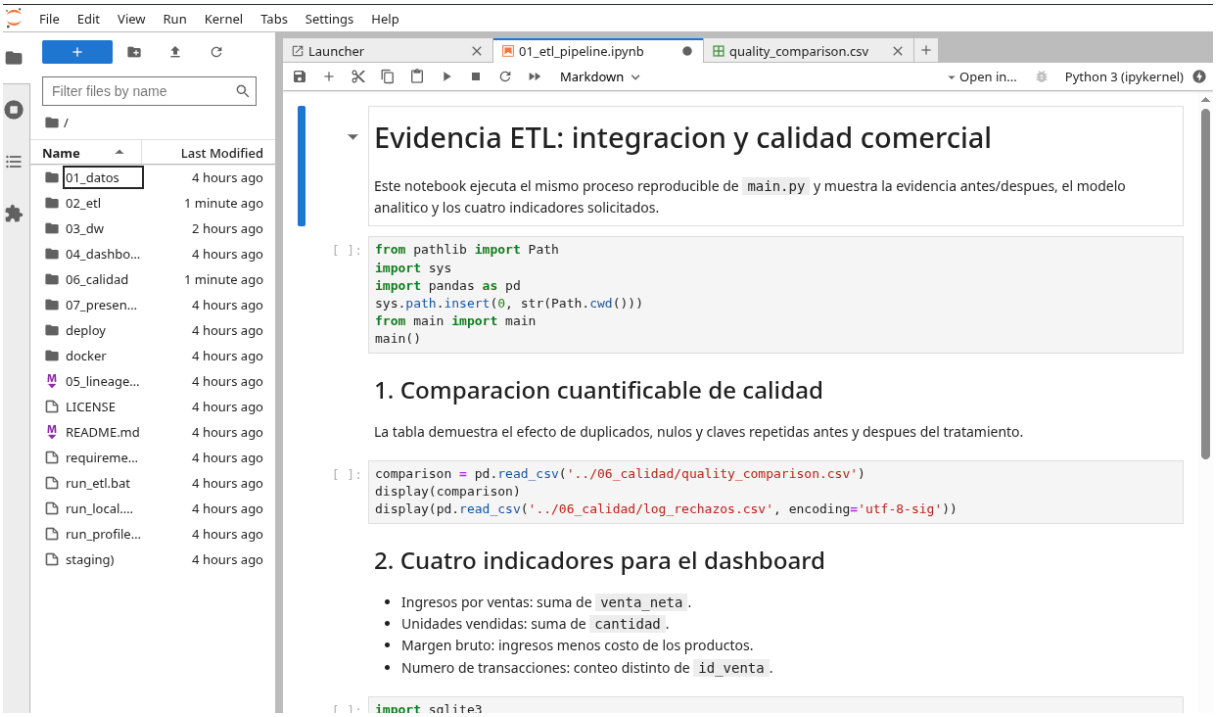
La estructura del repositorio separa datos, ETL, data warehouse, dashboard, calidad y presentación. Esa separación facilita la revisión por parte de terceros y permite reemplazar una capa sin afectar las demás. El informe y sus figuras se generan con 07_presentacion/build_informe.py, que consulta la base y los reportes de calidad para mantener las cifras sincronizadas.

Anexo E. Evidencias del proceso ETL y del dashboard en la nube

Las siguientes capturas documentan la ejecución del pipeline, la calidad de los datos, la carga en Neon y la publicación del dashboard en Apache Superset. Se añaden tres evidencias específicas de los cambios: el modelo con FactRechazos y las vistas nuevas, el dashboard ejecutivo con mapa-filtro y leyenda, y la gestión de pérdidas por rechazos y descuentos por categoría. Se incluyen como evidencia de trazabilidad del proyecto y conservan la numeración consecutiva de las figuras del informe.

Figura 12

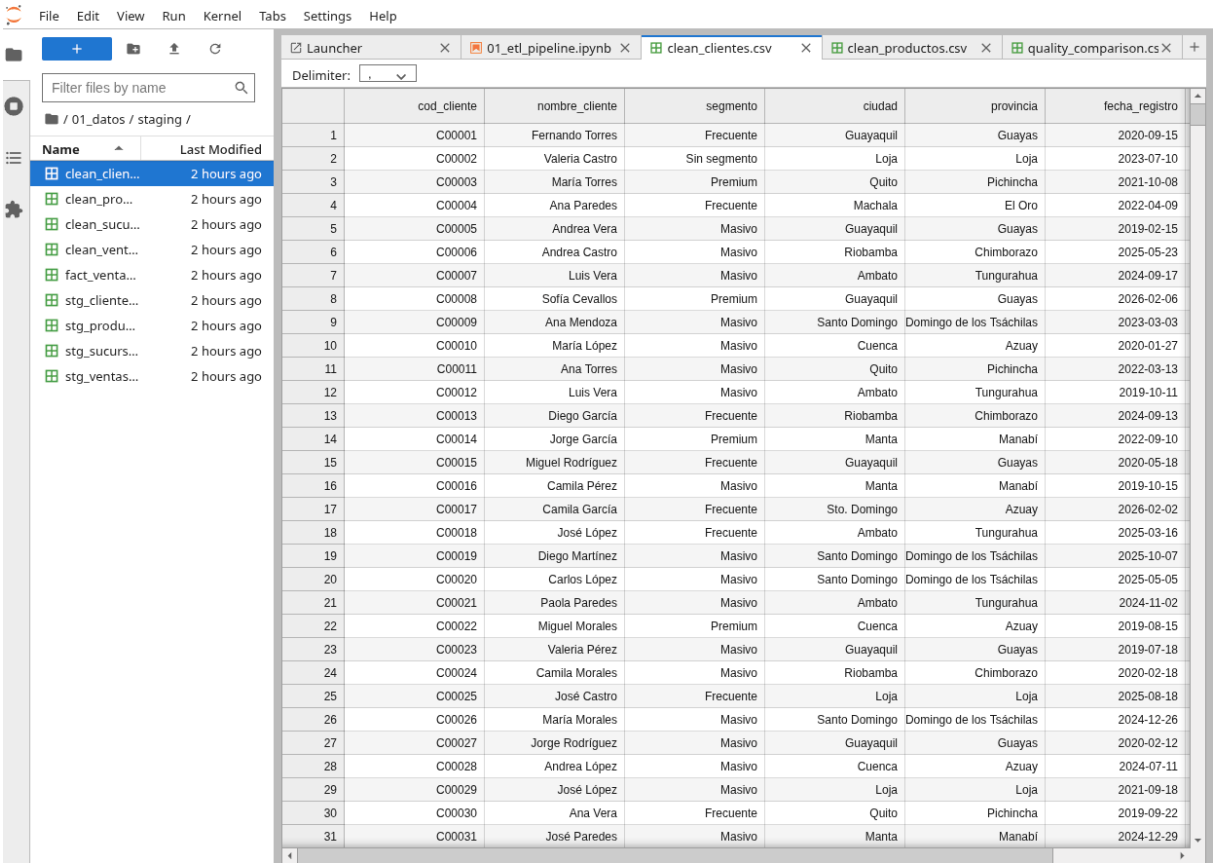
Notebook de Jupyter con la ejecución del pipeline ETL



Nota. Captura de la ejecución de 02_etl/01_etl_pipeline.ipynb en Jupyter. Elaboración propia.

Figura 13

Datos procesados en la etapa de staging

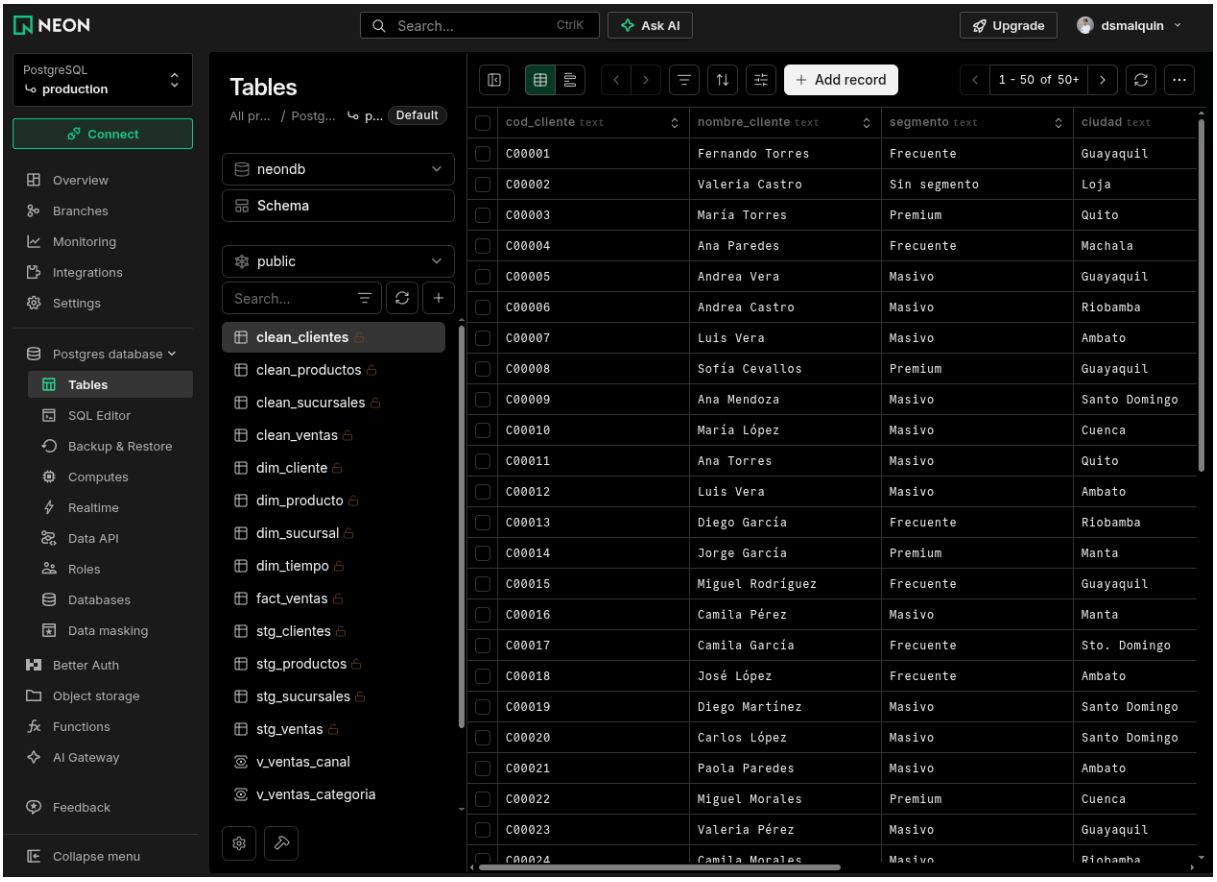


	cod_cliente	nombre_cliente	segmento	ciudad	provincia	fecha_registro
1	C00001	Fernando Torres	Frecuente	Guayaquil	Guayas	2020-09-15
2	C00002	Valeria Castro	Sin segmento	Loja	Loja	2023-07-10
3	C00003	María Torres	Premium	Quito	Pichincha	2021-10-08
4	C00004	Ana Paredes	Frecuente	Machala	El Oro	2022-04-09
5	C00005	Andrea Vera	Masivo	Guayaquil	Guayas	2019-02-15
6	C00006	Andrea Castro	Masivo	Riobamba	Chimborazo	2025-05-23
7	C00007	Luis Vera	Masivo	Ambato	Tungurahua	2024-09-17
8	C00008	Sofía Cevallos	Premium	Guayaquil	Guayas	2026-02-06
9	C00009	Ana Mendoza	Masivo	Santo Domingo	Domingo de los Tsáchilas	2023-03-03
10	C00010	María López	Masivo	Cuenca	Azuay	2020-01-27
11	C00011	Ana Torres	Masivo	Quito	Pichincha	2022-03-13
12	C00012	Luis Vera	Masivo	Ambato	Tungurahua	2019-10-11
13	C00013	Diego García	Frecuente	Riobamba	Chimborazo	2024-09-13
14	C00014	Jorge García	Premium	Manta	Manabí	2022-09-10
15	C00015	Miguel Rodríguez	Frecuente	Guayaquil	Guayas	2020-05-18
16	C00016	Camila Pérez	Masivo	Manta	Manabí	2019-10-15
17	C00017	Camila García	Frecuente	Sto. Domingo	Azuay	2026-02-02
18	C00018	José López	Frecuente	Ambato	Tungurahua	2025-03-16
19	C00019	Diego Martínez	Masivo	Santo Domingo	Domingo de los Tsáchilas	2025-10-07
20	C00020	Carlos López	Masivo	Santo Domingo	Domingo de los Tsáchilas	2025-05-05
21	C00021	Paola Paredes	Masivo	Ambato	Tungurahua	2024-11-02
22	C00022	Miguel Morales	Premium	Cuenca	Azuay	2019-08-15
23	C00023	Valeria Pérez	Masivo	Guayaquil	Guayas	2019-07-18
24	C00024	Camila Morales	Masivo	Riobamba	Chimborazo	2020-02-18
25	C00025	José Castro	Frecuente	Loja	Loja	2025-08-18
26	C00026	María Morales	Masivo	Santo Domingo	Domingo de los Tsáchilas	2024-12-26
27	C00027	Jorge Rodríguez	Masivo	Guayaquil	Guayas	2020-02-12
28	C00028	Andrea López	Masivo	Cuenca	Azuay	2024-07-11
29	C00029	José López	Masivo	Loja	Loja	2021-09-18
30	C00030	Ana Vera	Frecuente	Quito	Pichincha	2019-09-22
31	C00031	José Paredes	Masivo	Manta	Manabí	2024-12-29

Nota. Captura de 01_datos/staging/clean_clientes.csv generado por el ETL. Elaboración propia.

Figura 14

Tablas originales y transformadas cargadas en Neon



Nota. Captura de la consola de Neon con las tablas stg_*, clean_*, dim_* y fact_ventas. Elaboración propia.

Figura 15

Comparación de calidad antes y después del ETL

Entidad	Filas antes	Filas despues	Diferencia	Nulos antes	Nulos despues	Duplicados exactos antes	Duplicados exactos despues	Claves unicas antes	Claves unicas despues
clientes	1006	1000	6	10	8	6	0	1000	1000
sucursales	12	12	0	0	0	0	0	12	12
productos	123	120	3	5	5	3	0	120	120
ventas	6870	6800	70	27	0	20	0	3600	3587

Nota. Captura de 06_calidad/quality_comparison.csv. Elaboración propia.

Figura 16

Reglas de transformación y justificación

Reglas de transformación y justificación

Problema observado	Fuente	Regla aplicada	Criterio y resultado esperado
Duplicados exactos	Las cuatro fuentes	Eliminar duplicados exactos; en catálogos conservar una fila por clave natural	Evitar doble conteo. En ventas se conserva el grano línea y solo se elimina una fila idéntica
Fechas heterogéneas, vacías o inválidas	Clientes, sucursales y ventas	Parsear con `pandas.to_datetime`; excluir ventas sin fecha	La dimensión tiempo solo contiene fechas válidas y el dashboard puede agrupar por año/mes
Descuentos en decimal o porcentaje	Ventas	Valores mayores que 1 se dividen para 100	Todos los descuentos quedan entre 0 y 1 para calcular venta neta
Categorías con distinta escritura	Productos	Homologar mayúsculas, minúsculas y acentos a un catálogo canónico	Evitar que una misma categoría aparezca como varias series
Segmento, canal o método vacío	Clientes y ventas	Sustituir vacío por una categoría explícita de no especificado	No perder filas y hacer visible la incertidumbre en el dashboard
Claves foráneas inexistentes	Ventas	Registrar el rechazo y excluir la línea del hecho integrado	Impedir ventas huérfanas que distorsionen filtros y agregados
Costo de producto desconocido	Productos	No imputar cero; conservar costo y margen como desconocidos	No sobreestimar artificialmente el margen bruto
Tipos numéricos almacenados como texto	Ventas y productos	Convertir cantidad, precio, descuento y costo a numérico	Permitir cálculos reproducibles de ingresos y margen

Nota. Captura de 06_calidad/reglas_transformacion.md. Elaboración propia.

Figura 17

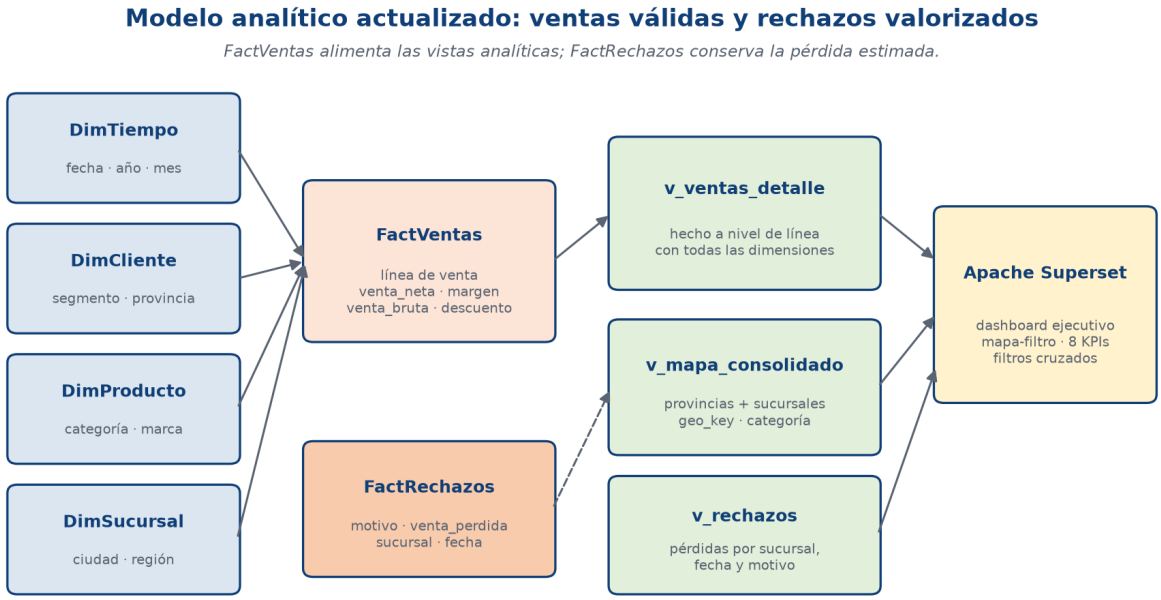
Dashboard Sales Analytics en Apache Superset



Nota. Captura del dashboard publicado en Apache Superset con datos de Neon. Elaboración propia.

Figura 18

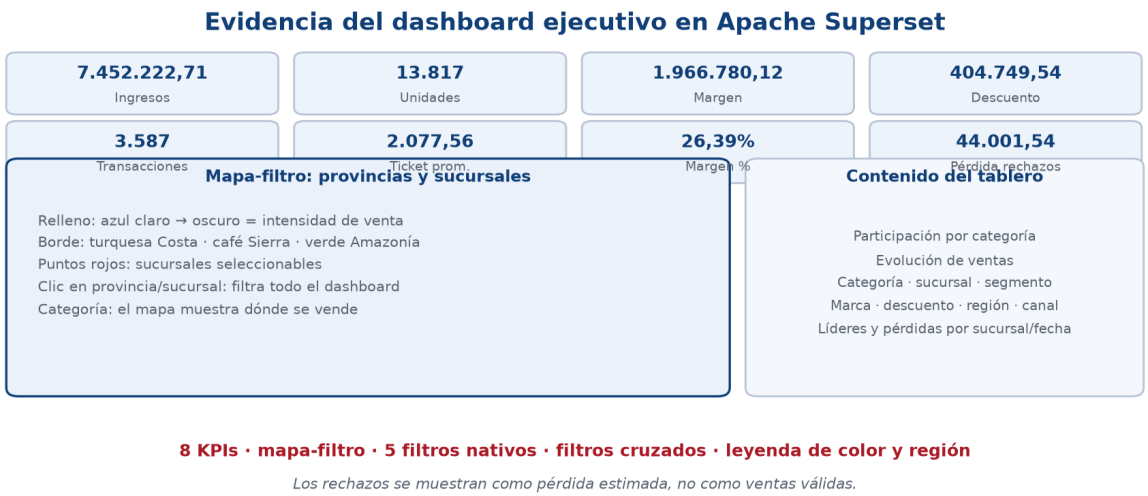
Modelo analítico actualizado con FactRechazos y vistas nuevas



Nota. Esquema propio a partir de 03_dw/schema.sql y 04_dashboard/kpi_views_postgres.sql. Incluye FactRechazos y las vistas v_ventas_detalle, v_mapa_consolidado y v_rechazos. Elaboración propia.

Figura 19

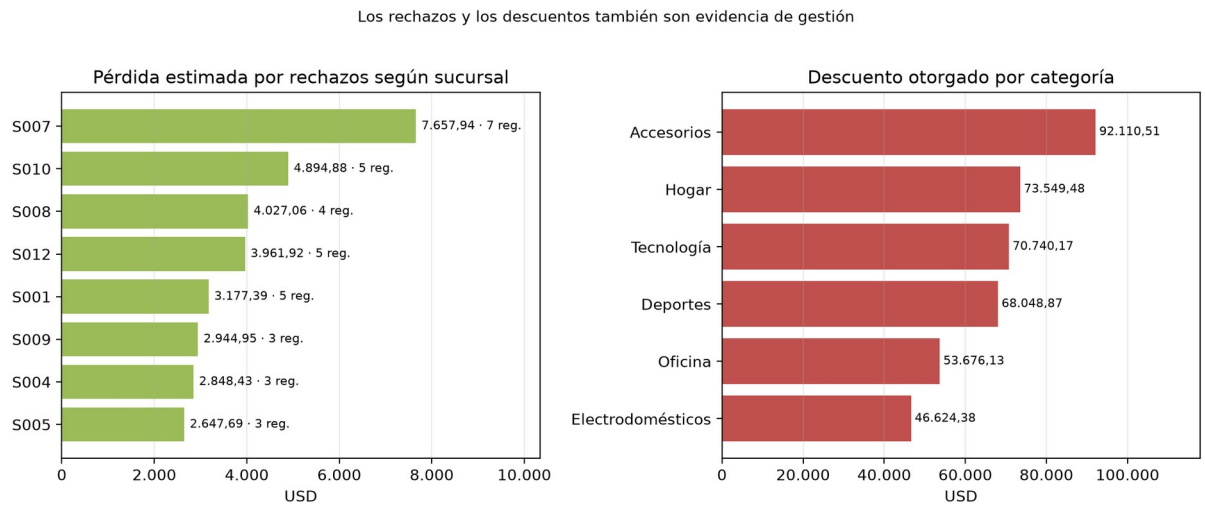
Dashboard ejecutivo actualizado: KPIs, mapa-filtro y leyenda



Nota. Evidencia del dashboard Sales Analytics en Apache Superset. Resume los ocho KPIs, el mapa-filtro y la leyenda de color y región. Elaboración propia.

Figura 20

Pérdidas por rechazos y descuentos por categoría



Nota. Evidencia de gestión: pérdida estimada por sucursal y descuento otorgado por categoría, ambos filtrables desde el dashboard. Elaboración propia.